第5章 神经网络

神经网络发展史

第一阶段

- 1943年, McCulloch和Pitts 提出第一个神经元数学模型, 即M-P模型, 并从原理上证明了人工神经网络能够计算任何算数和逻辑函数
- 1949年, Hebb 发表《The Organization of Behavior》一书, 提出生物神经元学习的机理, 即Hebb学习规则
- 1958年, Rosenblatt 提出感知机网络（Perceptron）模型和其学习规则
- 1960年, Widrow和Hoff提出自适应线性神经元（Adaline）模型和最小均方学习算法
- 1969年, Minsky和Papert 发表《Perceptrons》一书, 指出单层神经网路不能解决非线性问题, 多层网络的训练算法尚无希望. 这个论断导致神经网络进入低谷

神经网络发展史

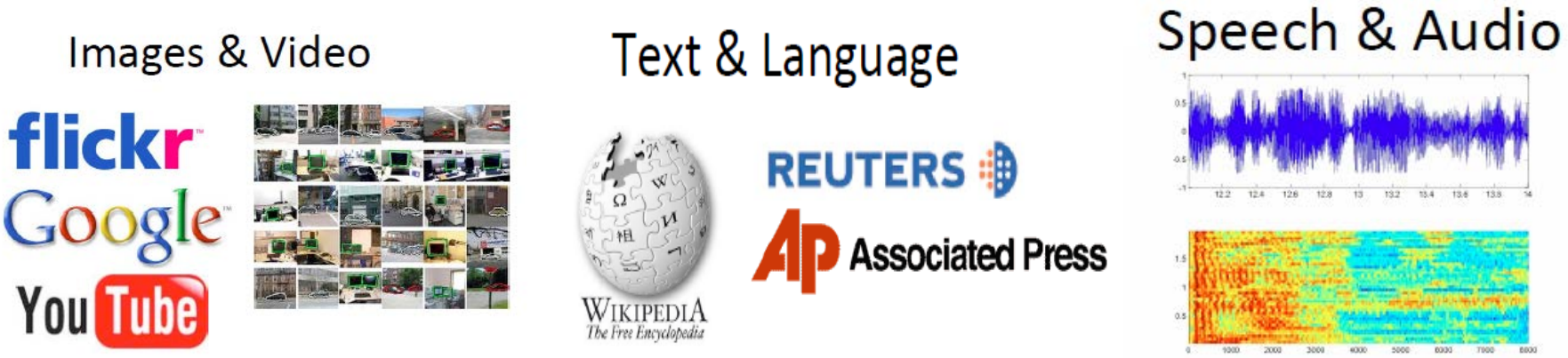
第二阶段

- 1982年, 物理学家Hopfield提出了一种具有联想记忆、优化计算能力的递归网络模型, 即Hopfield 网络
- 1986年, Rumelhart 等编辑的著作《Parallel Distributed Proceesing:Explorations in the Microstructures of Cognition》报告了反向传播算法
- 1987年, IEEE 在美国加州圣地亚哥召开第一届神经网络国际会议（ICNN）
- 90年代初, 伴随统计学习理论和SVM的兴起, 神经网络由于理论不够清楚, 试错性强, 难以训练, 再次进入低谷

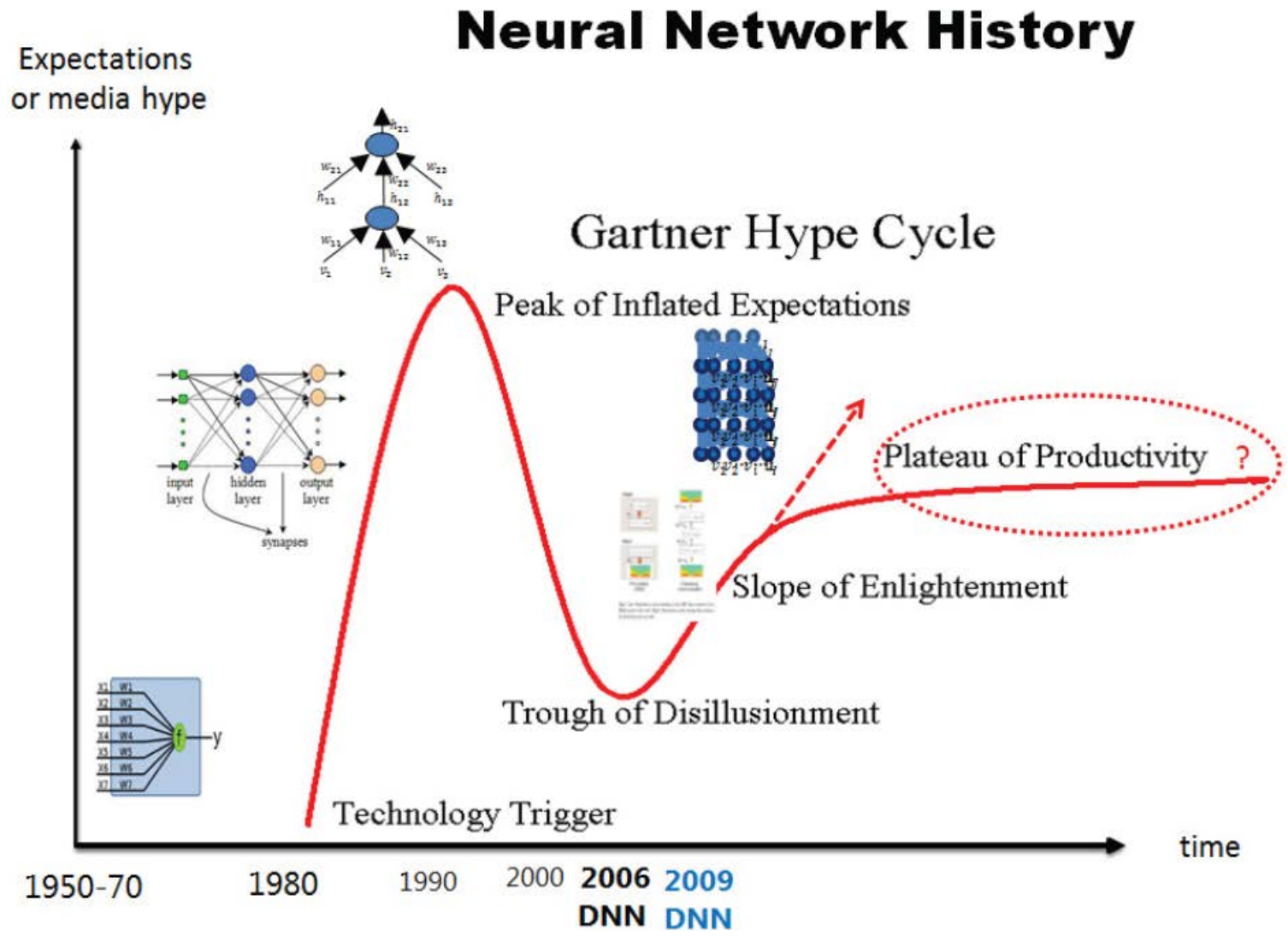
神经网络发展史

第三阶段

- 2006年, Hinton提出了深度信念网络(DBN), 通过“预训练+微调”使得深度模型的最优化变得相对容易
- 2012年, Hinton 组参加ImageNet 竞赛, 使用 CNN 模型以超过第二名10个百分点的成绩夺得当年竞赛的冠军
- 伴随云计算、大数据时代的到来，计算能力的大幅提升，使得深度学习模型在计算机视觉、自然语言处理、语音识别等众多领域都取得了较大的成功



神经网络发展史


Neural Network History
Expectations or media hype
Gartner Hype Cycle
Peak of Inflated Expectations
Plateau of Productivity ?
Slope of Enlightenment
Trough of Disillusionment
Technology Trigger
time
input layer
hidden layer
output layer
synapses
1950-70
1980
1990
2000
2006 DNN
2009 DNN

第5章 神经网络

主要内容

- 5.1 神经元模型
- 5.2 感知机与多层网络
- 5.3 误差逆传播算法
- 5.4 全局最小与局部最小
- 5.5 其他常见神经网络
- 5.6 深度学习

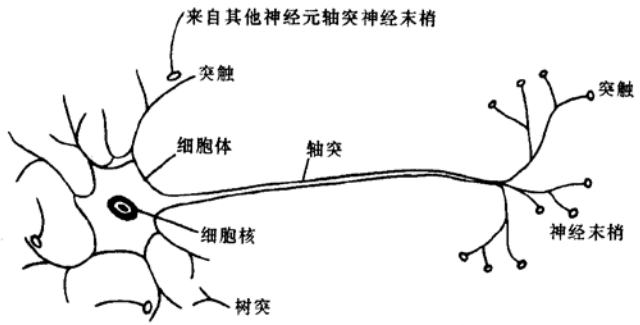
5.1 神经元模型

神经网络的定义

“神经网络是由具有适应性的简单单元组成的广泛并行互联的网络，它的组织能够模拟生物神经系统对真实世界物体所作出的反应”

[Kohonen, 1988]

- 机器学习中的神经网络通常是指“神经网络学习”或者机器学习与神经网络两个学科的交叉部分
- 神经元模型即上述定义中的“简单单元”是神经网络的基本成分
- 生物神经网络：每个神经元与其他神经元相连，当它“兴奋”时，就会向相连的神经云发送化学物质，从而改变这些神经元内的电位；如果某神经元的电位超过一个“阈值”，那么它就会被激活，即“兴奋”起来，向其它神经元发送化学物质



M-P 神经元模型 [McCulloch and Pitts, 1943]

5.1 神经元模型

- 输入：来自其他n个神经云传递过来的输入信号
- 处理：输入信号通过带权重的连接进行传递, 神经元接受到总输入值将与神经元的阈值进行比较
- 输出：通过激活函数的处理以得到输出

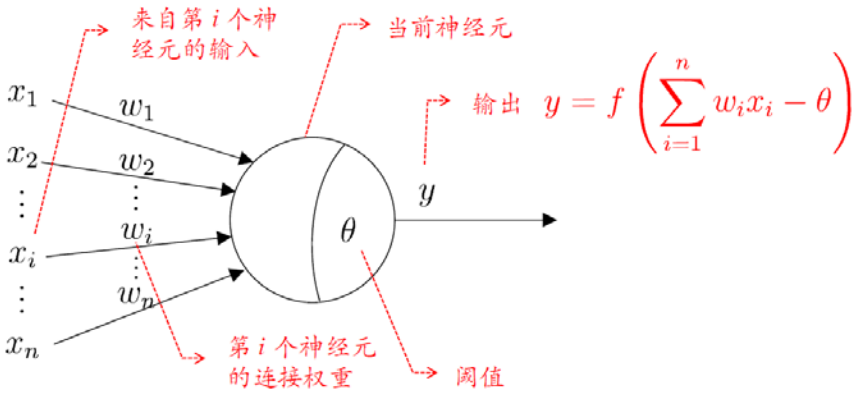


图 5.1 M-P 神经元模型

5.1 神经元模型

激活函数

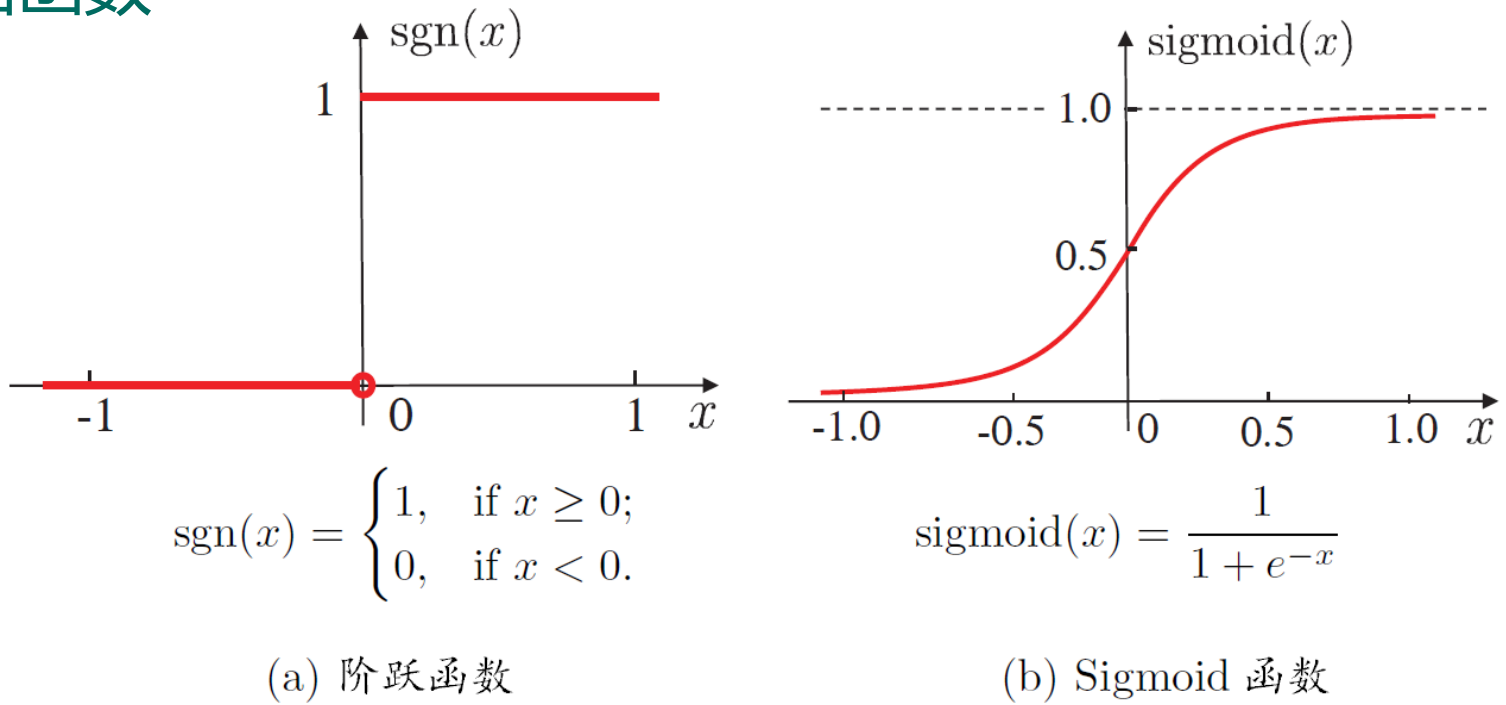
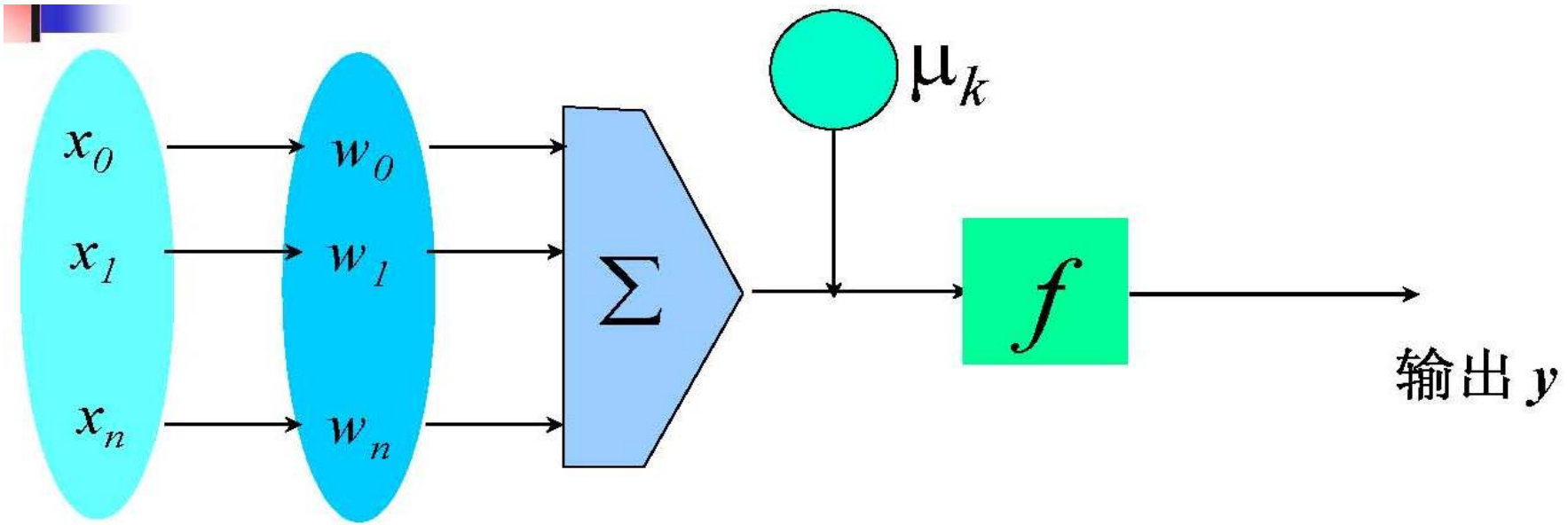


图 5.2 典型的神经元激活函数

- 理想激活函数是阶跃函数, 0表示抑制神经元而1表示激活神经元
- 阶跃函数具有不连续、不光滑等不好的性质, 常用的是 Sigmoid 函数

神经元

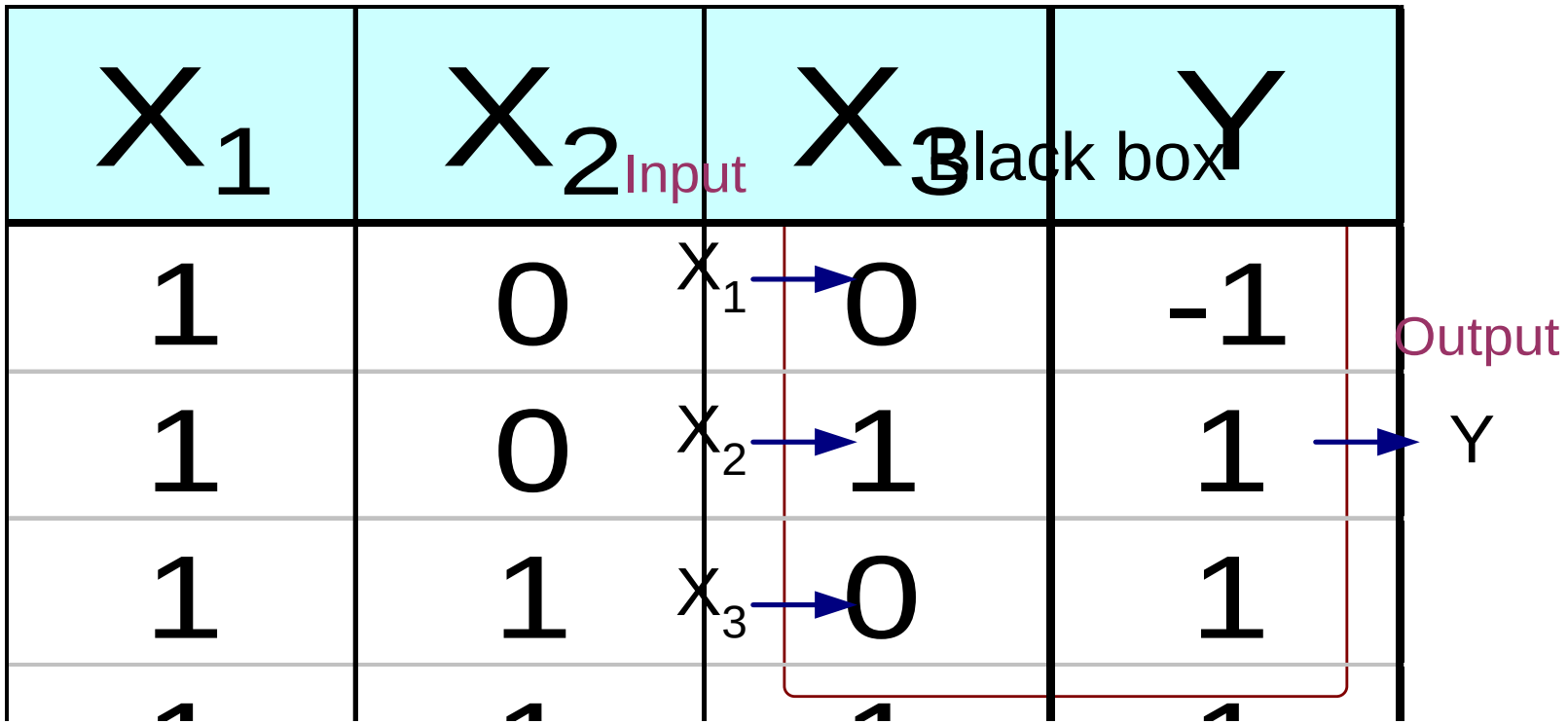


输入向量 x　　权重向量 w　　加权和　　赋活函数

- **n**维的输入向量**x**被映射到变量**y**,通过标积 和非线性函数映射

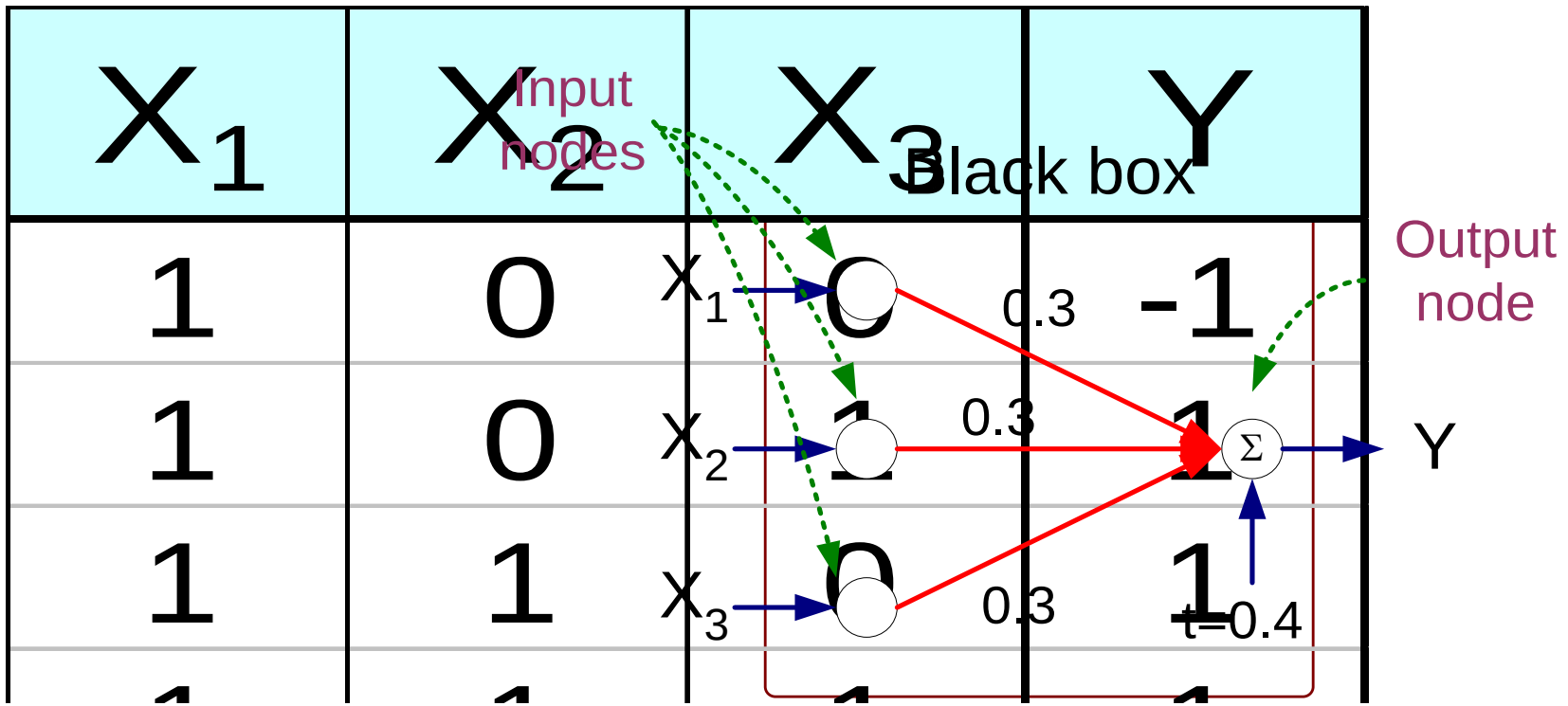
例子

X_1	X_2	X_3	Y
1	0	0	-1
1	0	1	1
1	1	0	1
1	1	1	1



Output Y is 1 if at least two of the three inputs are equal to 1.

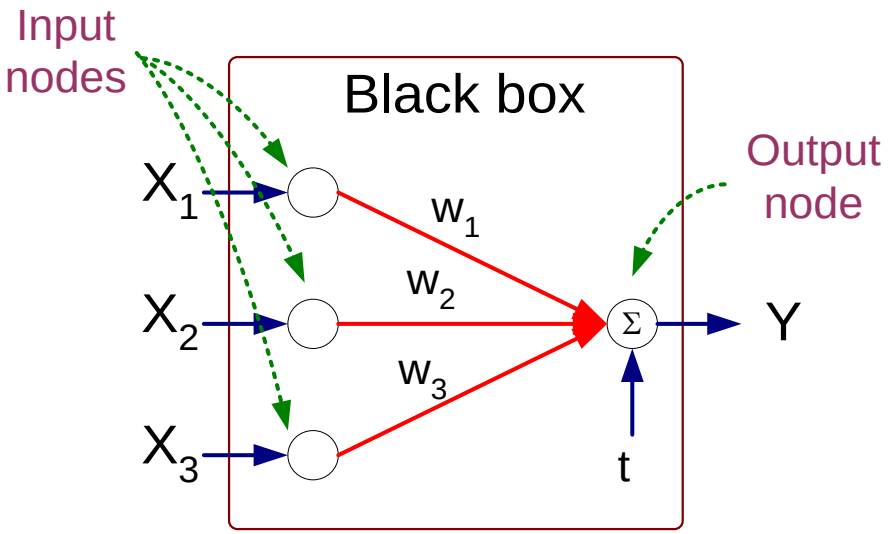
X_1	X_2	X_3	Y
1	0	0	-1
1	0	1	1
1	1	0	1
1	1	1	1



$$Y = sign(0.3X_1 + 0.3X_2 + 0.3X_3 - 0.4)$$

$$\text{where } sign(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ -1 & \text{if } x < 0 \end{cases}$$

- Model is an assembly of inter-connected nodes and weighted links
- Output node sums up each of its input value according to the weights of its links
- Compare output node against some threshold t



感知器模型

$$Y = sign(\sum_{i=1}^{d} w_i X_i - t)$$

$$= sign(\sum_{i=0}^{d} w_i X_i)$$

神经网络模型

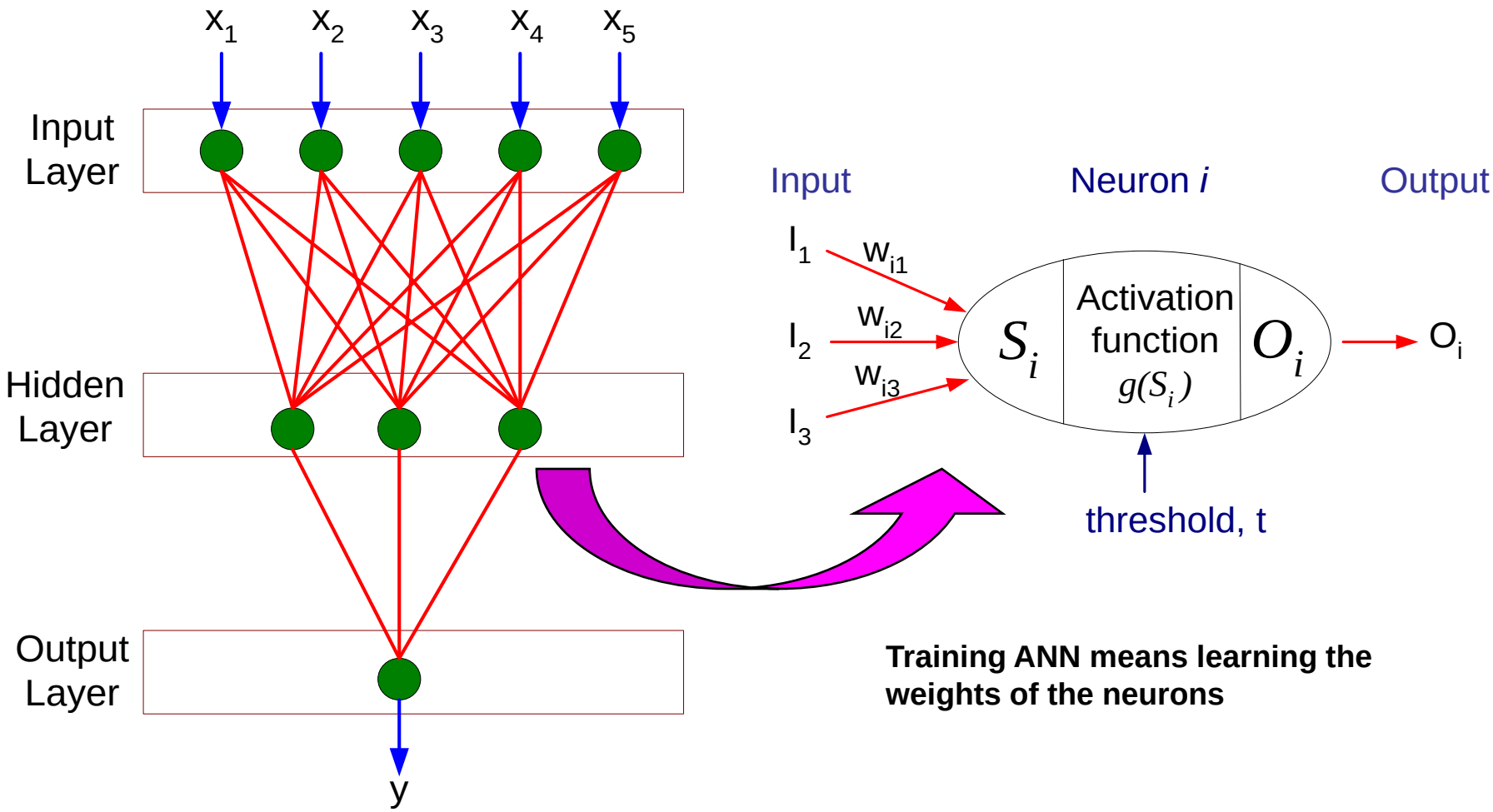
神经网络模型很多，典型的模型有感知器、BP神经网络、Hopfied网络、卷积神经网络、递归神经网络等等，这里主要介绍单神经元感知器、单层感知器、BP神经网络和Rprop神经网络。

感知器（Perceptron）是第一个完整的人工神经网络，最早是由美国心理学家Rosenblatt于1958年提出的，是一种典型的前向神经网络，可分为单神经元感知器、单层感知器和多层感知器。通常说的感知器不能解决线性不可分问题是指单神经元感知器和单层感知器，而多层感知器（Multilayer Perceptrons，MLP）可以解决线性不可分问题。由于多层感知器利用反向传播算法（Back-Propagation algorithm）进行训练，所以多层感知器也常常被称作BP网络。

神经网络训练

- 训练的最终目标
  - 获得一个权重集使得训练数据中的所有元组都能被正确的分类
- 步骤
  - 用任意的值初始化权
  - 向前传播输入
    - 对任意单元
    - 每个单元的净输入用它输入的线性组合计算
    - 使用赋活函数计算输出值
    - 计算错误
    - 更新权和基

ANN通用结构



Training ANN means learning the weights of the neurons

单神经元感知器

Rosenblatt感知器是典型的单层感知器，建立在一个非线性神经元上，与神经元的McCulloch-Pitts（M-P）模型十分相似，是用于线性可分模式（存在一个超平面可以把数据分成两类）的最简单的神经网络模型。由于只有一个神经元，所以只能完成两类的模式分类。

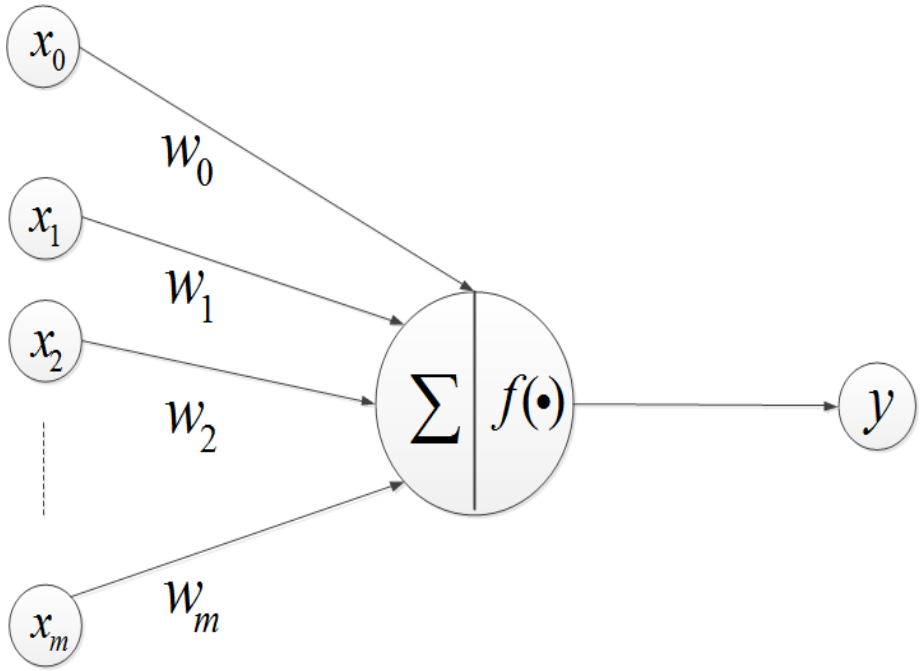


图 单神经元感知器

在神经元模型的基础上，选择符号函数作为激活函数，由于这里只有一个神经元，可以省略下标k。第t步的训练数据集为 $D_t=\{(\mathbf{x}_n(t),d_n(t)),1\le n\le N\}$，$\mathbf{x}_n(t)=[1,x_{n1}(t),x_{n2}(t),\cdots,x_{nm}(t)]^T$ 权值向量为 $\mathbf{w}(t)=[w_0(t),w_1(t),w_2(t),\cdots,w_m(t)]^T$ $d_n(t)$代表期望输出取值为1或−1，输出为

$$y_n(t)=f(\mathbf{w}_n^T(t)\mathbf{x}_n(t))=sgn(\mathbf{w}_n^T(t)\mathbf{x}_n(t))=\begin{cases}1, & \mathbf{w}_n^T(t)\mathbf{x}_n(t)\ge 0\\ -1, & \mathbf{w}_n^T(t)\mathbf{x}_n(t)<0\end{cases}$$

单神经元感知器的训练目标是找到一个 $\mathbf{w}^*$，使得如果$d_n=1$，$\mathbf{w}^*\mathbf{x}\ge 0$（类1），否则 $\mathbf{w}^*\mathbf{x}<0$ （类2）。利用最小均方误差规则进行训练的过程为：

第一步：初始化网络权重为小的随机数，$t=0$ 。

第二步：输入第t步的训练数据，计算单神经元感知器的输出与期望输出的误差，调整权重：

$$\Delta w_i(t) = \eta \sum_{n=1}^{N} (d_n(t) - y_n(t))\, x_{ni}(t) \quad i = 0,1,2,\cdots,m$$

$$w_i(t+1) = w_i(t) + \Delta w_i(t)$$

第三步：重复步骤二，直至训练数据集的误差小于预先设定的阈值或达到最大的迭代次数。

11.3.2 单层感知器

建立在一个神经元上的感知器只能完成两类的模式分类，通过扩展感知器的输出层可以使感知器包括不止一个神经元，相应地可以进行多类别的分类，这就是单层感知器。

设单层感知器由K个神经元构成，当样本属于第k类时，第k个神经元的输出为1，其余神经元的输出为0，以此进行多分类。

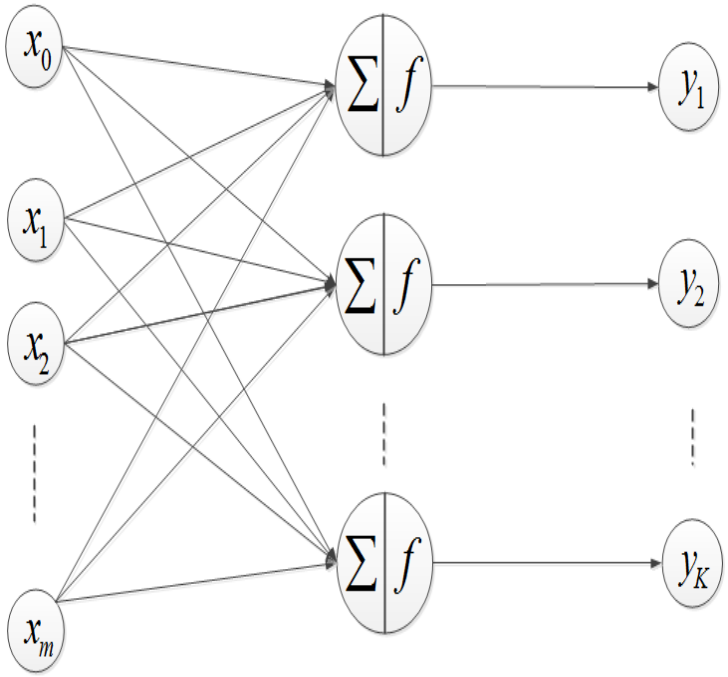


图 单层感知器

与单神经元感知器相似，设第t步的训练数据集为$D_t=\{(\mathbf{x}_n(t),\mathbf{d}_n(t)),1\le n\le N\}$，输入为$\mathbf{x}_n(t)=[1,x_{n1}(t),x_{n2}(t),\cdots,x_{nm}(t)]^T$ 则第K个神经元的权值向量为$\mathbf{w}_k(t)=[w_{k0}(t),w_{k1}(t),w_{k2}(t),\cdots,w_{km}(t)]^T \quad k=1,2,\cdots,K$，另外 $\mathbf{d}_n(t)=[d_{n1}(t),d_{n1}(t),\cdots,d_{nK}(t)]^T$表示$K$个神经元的期望输出，$f(g)$是激活函数，$\mathbf{y}_n(t)=[y_{n1}(t),y_{n1}(t),\cdots,y_{nK}(t)]^T$表示$K$个神经元的实际输出，其中，$y_{nk}(t)=f(\mathbf{w}_k(t)^T\mathbf{x}_n(t)) \quad k=1,2,\cdots,K$，则利用最小均方误差规则进行训练的过程如下：

第一步：初始化网络权重为小的随机数，$t=0$。

第二步：输入第t步的训练数据，计算单层感知器的输出与期望输出的误差，其中，第k个神经元的误差函数为：

$$E_k(t)=\frac{1}{2}\sum_{n=1}^{N}(d_{nk}(t)-y_{nk}(t))^2 \quad k=1,2,\cdots,K$$

接着调整权重：

$$\Delta w_{ki}(t)=-\eta\frac{\partial E(t)}{\partial w_{ki}}=\eta\sum_{n=1}^{N}(d_n(t)-y_n(t))\,x_{ni}(t) \qquad i=0,1,2,\cdots,m$$

$$w_{ki}(t+1)=w_{ki}(t)+\Delta w_{ki}(t) \qquad k=1,2,\cdots,K$$

第三步：重复步骤二，直至训练数据集的误差小于预先设定的阈值或达到最大的迭代次数。

5.2 感知机与多层网络

感知机

- 感知机由两层神经元组成，输入层接受外界输入信号传递给输出层，输出层是M-P神经元（阈值逻辑单元）
- 感知机能够容易地实现逻辑与、或、非运算
  - “与” $x_1 \wedge x_2$：令 $w_1 = w_2 = 1, \theta = 0$，则 $y = f(1 \cdot x_1 + 1 \cdot x_2 - 2)$，仅在 $x_1 = x_2 = 1$ 时，$y = 1$.
  - “或” $x_1 \vee x_2$：令 $w_1 = w_2 = 1, \theta = 0.5$，则 $y = f(1 \cdot x_1 + 1 \cdot x_2 - 2)$，仅在 $x_1 = 1$ 或者 $x_2 = 1$ 时，$y = 1$.
  - “非” $\neg x_1$：令 $w_1 = -0.6, w_2 = 0, \theta = -0.5$，则 当 $x_1 = 1$ 时，$y = 0$；当 $x_1 = 0$，$y = 1$.

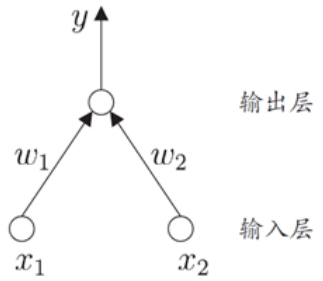


图 5.3 两个输入神经元的感知机网络结构示意图

感知机模型

感知机模型(两层神经元)

$$y = f(\sum_{i=1}^{n} \omega_i x - \theta) = f(\omega^T x - \theta)$$

超平面

$$\omega^T x - \theta = 0$$

$$y = f(\omega^T x - \theta) = \begin{cases} 1 & \omega^T x - \theta \geq 0 \\ -1 & \omega^T x - \theta < 0 \end{cases}$$

感知机模型

感知机模型学习策略

$$L(\omega,\theta)=\sum_{x\in M}(\hat{y}-y)(\omega^T x-\theta)$$

其中M 为误分类集合。

- 算法

$$\min_{\omega,\theta} L(\omega,\theta)=\min_{\omega,\theta}\sum_{x_i\in M}(\hat{y}_i-y_i)(\omega^T x_i-\theta)$$

- 利用随机梯度下降策略

$$w\leftarrow w+\Delta w$$

$$\Delta w=\eta(y_i-\hat{y}_i)x_i$$

5.2 感知机与多层网络

感知机学习

- 给定训练数据集, 权重$w_i(i=1,2,\cdots,n)$ 与阈值 θ 可以通过学习得到

- 感知机学习规则

对训练样例$(\boldsymbol{x},y)$ 若当前感知机的输出为$\hat{y}$, 则感知机权重调整规则为：

$$w_i \leftarrow w_i + \triangle w_i$$
$$\triangle w_i = \eta(y-\hat{y})x_i$$

其中$\eta \in (0,1)$称为学习率

若感知机对训练样例 $(\boldsymbol{x},y)$预测正确, 则感知机不发生变化；否则根据错误程度进行权重的调整.

感知机学习算法

感知器学习规则的训练步骤：

- （1）权值初始化
- （2）输入样本对
- （3）计算输出
- （4）根据感知器学习规则调整权值
- （5）返回到步骤(2)输入下一对样本，周而复始直到对所有样本，感知器的实际输出与期望输出相等。

$$w_i \leftarrow w_i + \triangle w_i$$

$$\triangle w_i = \eta(y - \hat{y})x_i$$

例一　单计算节点感知器，3个输入。给定3对训练样本对如下：

$X^1=(-1，1，-2，0)^T$　　$d^1=-1$

$X^2=(-1，0，1.5，-0.5)^T$　　$d^2=-1$

$X^3=(-1，-1，1，0.5)^T$　　$d^3=1$

设初始权向量$W(0)=(0.5,1,-1,0)^T$，　$\eta=0.1$。注意，输入向量中第一个分量x_0恒等于-1，权向量中第一个分量为阈值，试根据以上学习规则训练该感知器。

解：第一步 输入X^1，得

$$W^T(0)X^1=(0.5,1,-1,0)(-1,1,-2,0)^T=2.5$$

$$o^1(0)=sgn(2.5)=1$$

$$W(1)= W(0)+ \eta\,[d^1- o^1(0)]\,X^1$$

$$=(0.5,1,-1,0)^T+0.1(-1-1)(-1,1,-2,0)^T$$

$$=(0.7,0.8,-0.6,0)^T$$

第二步 输入X^2，得

$$W^T(1)X^2=(0.7,0.8,-0.6,0)(-1,0,1.5,-0.5)^T=-1.6$$

$$o^2(1)=sgn(-1.6)=-1$$

$$W(2)= W(1)+ \eta [d^2- o^2(1)] X^2$$

$$=(0.7,0.8,-0.6,0)^T+0.1[-1-(-1)](-1,0,1.5,-0.5)^T$$

$$=(0.7,0.8,-0.6,0)^T$$

由于$d^2= o^2(1)$，所以$W(2)= W(1)$。

第三步 输入X^3，得

$$W^T(2)X^3=(0.7,0.8,-0.6,0)(-1,-1,1,0.5)^T=-2.1$$

$$O^3(2)=sgn(-2.1=-1$$

$$W(3)= W(2)+ \eta\,[d^3- o^3(2)]\,X^3$$

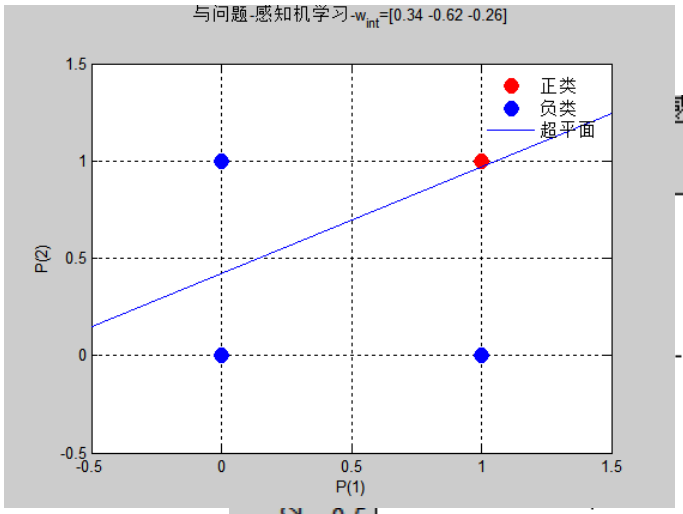
$$=(0.7,0.8,-0.6,0)^T+0.1[1-(-1)](-1,-1,1,0.5)^T$$

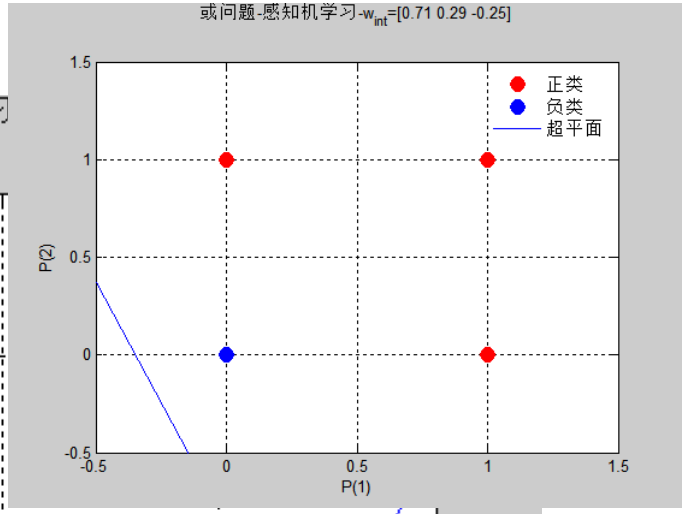
$$=(0.5,0.6,-0.4,0.1)^T$$

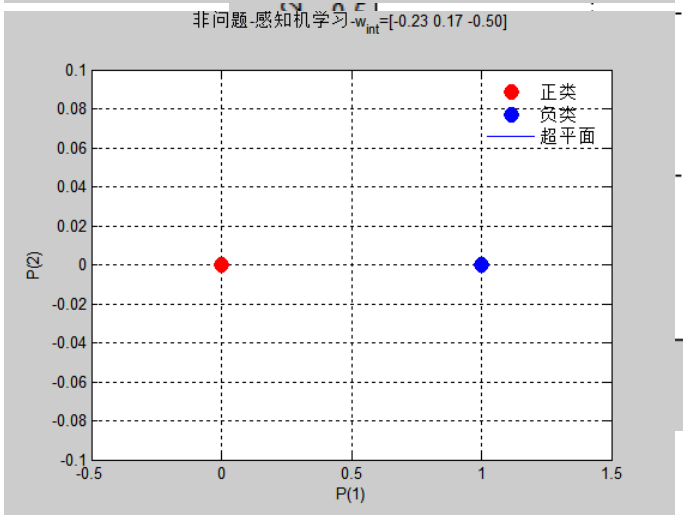
第四步 返回到第一步，继续训练直到$d^p- o^p=0$，$p=1,2,3$。

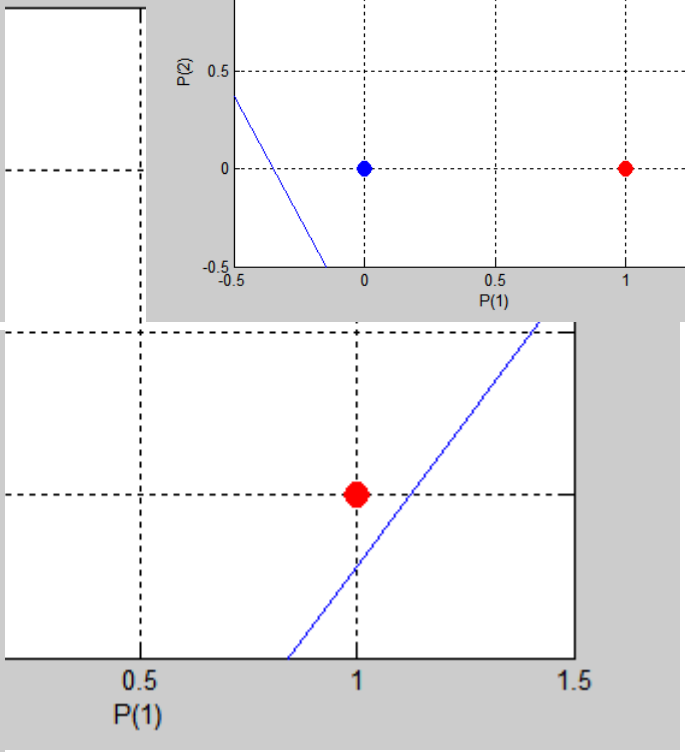
5.2 感知机与多层网络

感知机求解异、或、非问题









5.2 感知机与多层网络

感知机求解异、或、非问题

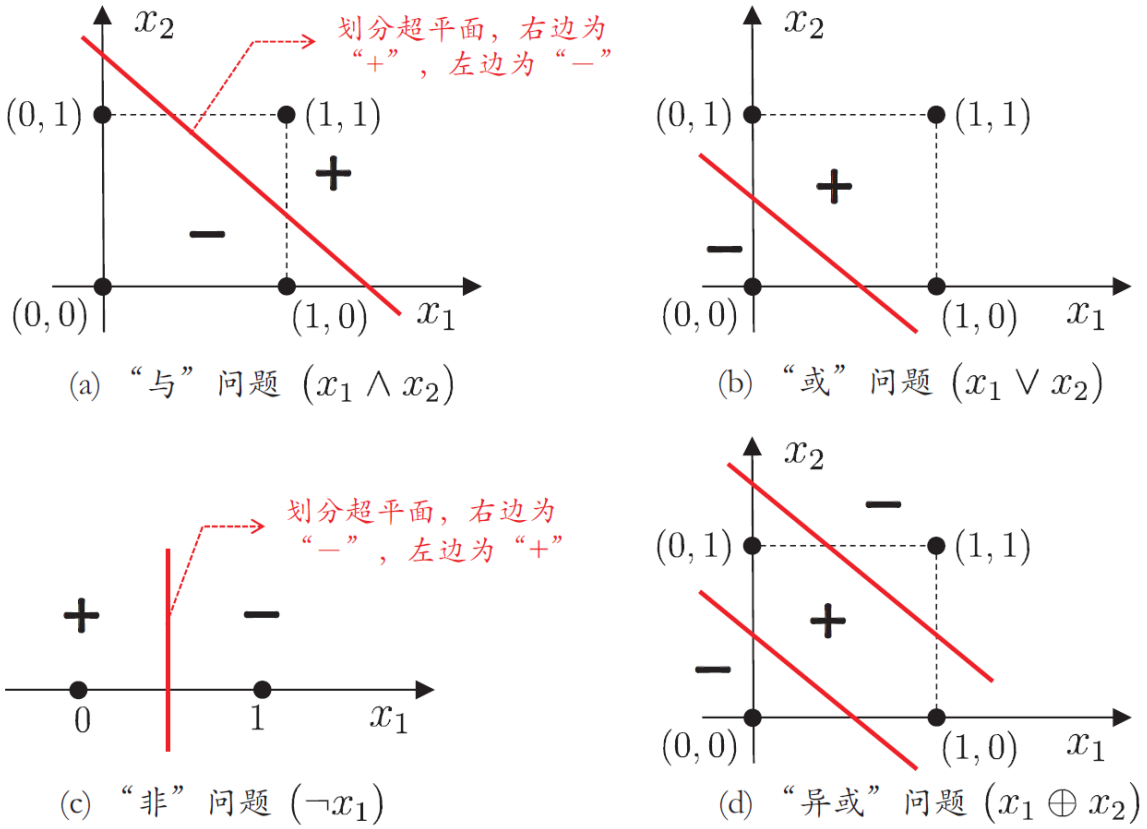


图 5.4 线性可分的“与”“或”“非”问题与非线性可分的“异或”问题

- 感知器收敛定理证明了如果训练数据是线性可分的，那么上述算法在有限步内收敛。对于线性不可分的情况，在训练过程中很有可能出现震荡，无法保证算法收敛。因此通常说感知器不能解决线性不可分问题。

练习题5.1

给定如下数据集

编号	X1	X2	Y
1	3	3	+
2	4	3	+
3	1	1	-

试用感知机学习算法的原始形式求感知机模型 f(x)=sign(wx+b)，其中初值 w=(0,0),b=0,学习率 η=1。

5.2 感知机与多层网络

感知机学习能力

- 若两类模式线性可分，则感知机的学习过程一定会收敛；否则感知机的学习过程将会发生震荡

  [Minsky and Papert, 1969]

- 单层感知机的学习能力非常有限，只能解决线性可分问题

- 事实上，与、或、非问题是线性可分的，因此感知机学习过程能够求得适当的权值向量. 而异或问题不是线性可分的，感知机学习不能求得合适解

- 对于非线性可分问题，如何求解？

  多层感知机

5.2 感知机与多层网络

多层感知机

- 解决异或问题的两层感知机

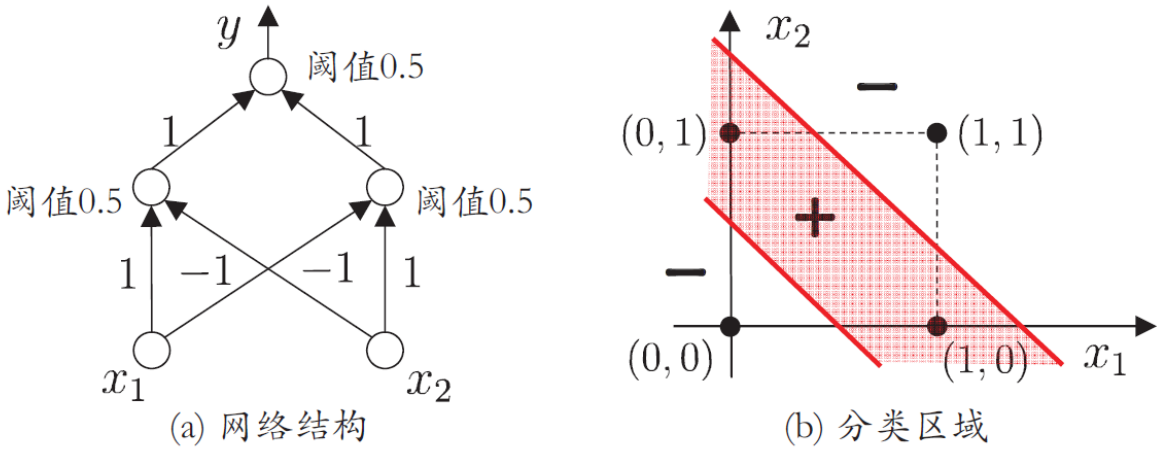
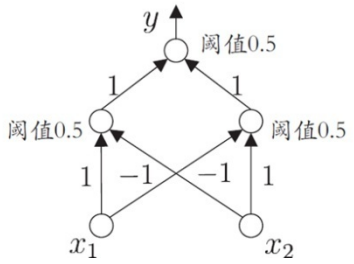


图 5.5　能解决异或问题的两层感知机

- 输出层与输入层之间的一层神经元, 被称之为隐层或隐含层, 隐含层和输出层神经元都是具有激活函数的功能神经元

练习题5.2



对如下数据集根据上面的两层感知机模型，计算混淆矩阵和分类准确率。

编号	X1	X2	Y
1	1	0	0
2	0	1	0
3	0	0	1
4	1	1	1

BP神经网络

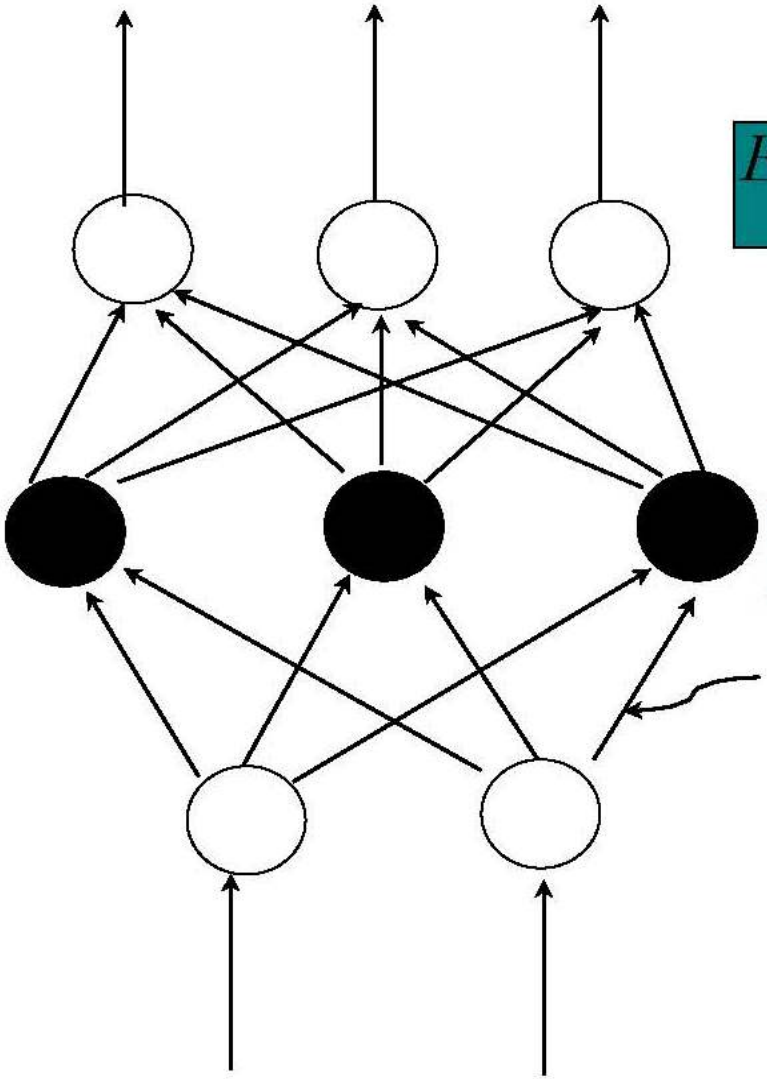
- BP神经网络是1986年由以Rumelhart和McCelland为首的科学小组提出的。它是单层感知器的推广，包括输入层、一个或多个隐藏层、输出层，隐藏层的存在使得BP神经网络可以解决线性不可分问题。BP神经网络通过误差反向传播算法进行训练，它是目前应用最广的神经网络模型。

5.2 感知机与多层网络

多层前馈神经网络

- 定义：每层神经元与下一层神经元全互联，神经元之间不存在同层连接也不存在跨层连接
- 前馈：输入层接受外界输入，隐含层与输出层神经元对信号进行加工，最终结果由输出层神经元输出
- 学习：根据训练数据来调整神经元之间的“连接权”以及每个功能神经元的“阈值”
- 多层网络：包含隐层的网络

多层前向神经网络



$$Err_j = O_j(1-O_j)\sum_k Err_k w_{jk}$$

$$\theta_j = \theta_j + (l)Err_j$$

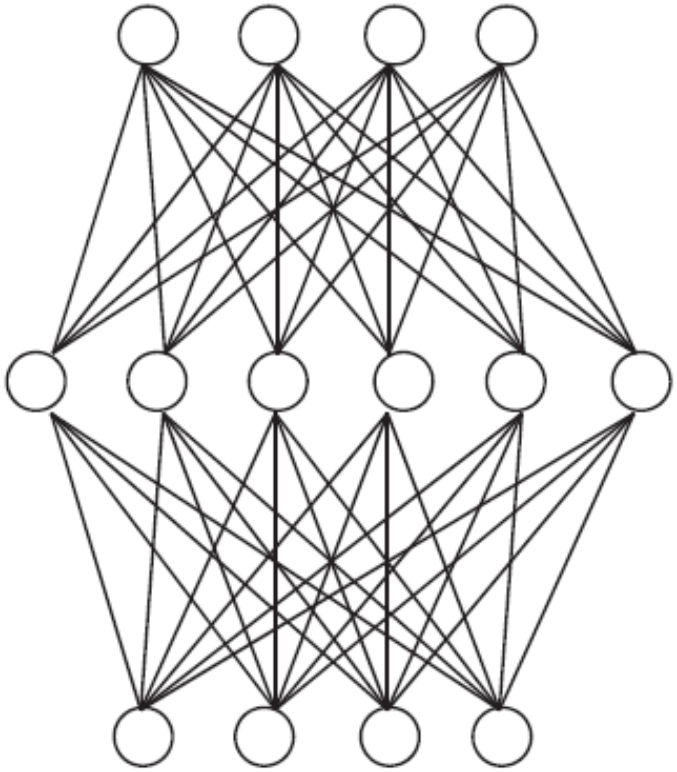
$$w_{ij} = w_{ij} + (l)Err_j O_i$$

$$Err_j = O_j(1-O_j)(T_j - O_j)$$

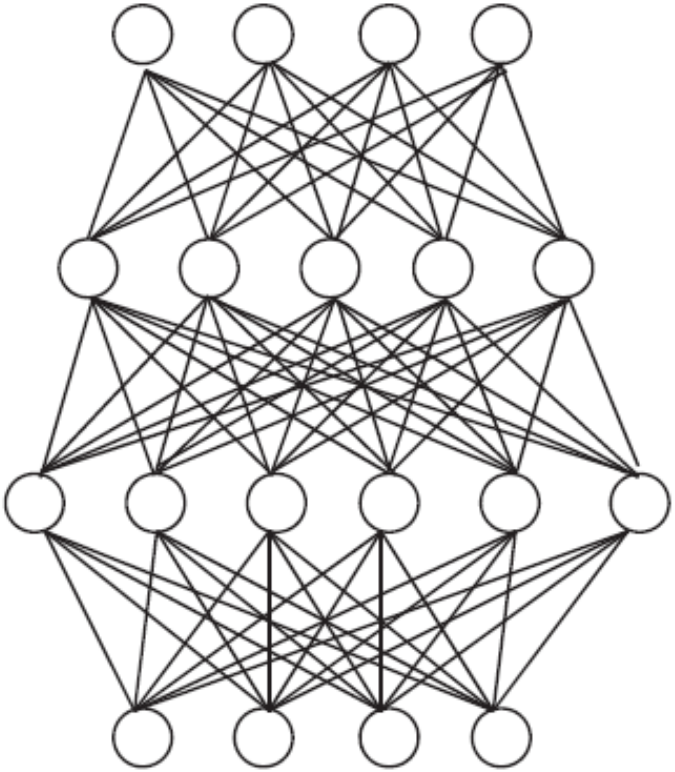
$$O_j = \frac{1}{1+e^{-I_j}}$$

$$I_j = \sum_i w_{ij} O_i + \theta_j$$

多层网络



(a) 单隐层前馈网络



(b) 双隐层前馈网络

多层神经网络如何工作的?

- 网络的**输入**对应于每个训练元组测量的属性
- 输入同时提供给称为**输入层**的单元层
- 然后加权提供给称为**隐藏层**的类神经元的第二层
- 隐藏层的数量是任意的，尽管实践中通常只用一层
- 最后的一个隐藏层的加权输出作为构成**输出层**的单元的输入。输出层发布给给定元组的网络预测。
- 网络是前馈的(**feed-forward**),如果其权都不回送到输入单元，或前一层的输出单元。
- 从统计学的观点来讲,他们进行非线性回归 : Given enough hidden units and enough training samples, they can closely approximate any function

定义网络拓扑结构

- **首先确定网络拓扑结构:** 输入层的单元数、隐藏层数 (if > 1), 、每个隐藏层的单元数和输出层的单元数
- 规范化每个属性的测量输入值，使其落在 [0.0—1.0]。
- 每个单元初始化为 0
- 对于分类，如果多于两个类，则每个类使用一个输出单元
- Once a network has been trained and its accuracy is **unacceptable**, repeat the training process with a *different network topology* or a *different set of initial weights*

5.3 误差逆传播算法

误差逆传播算法（Error BackPropagation, 简称BP）是最成功的训练多层前馈神经网络的学习算法.

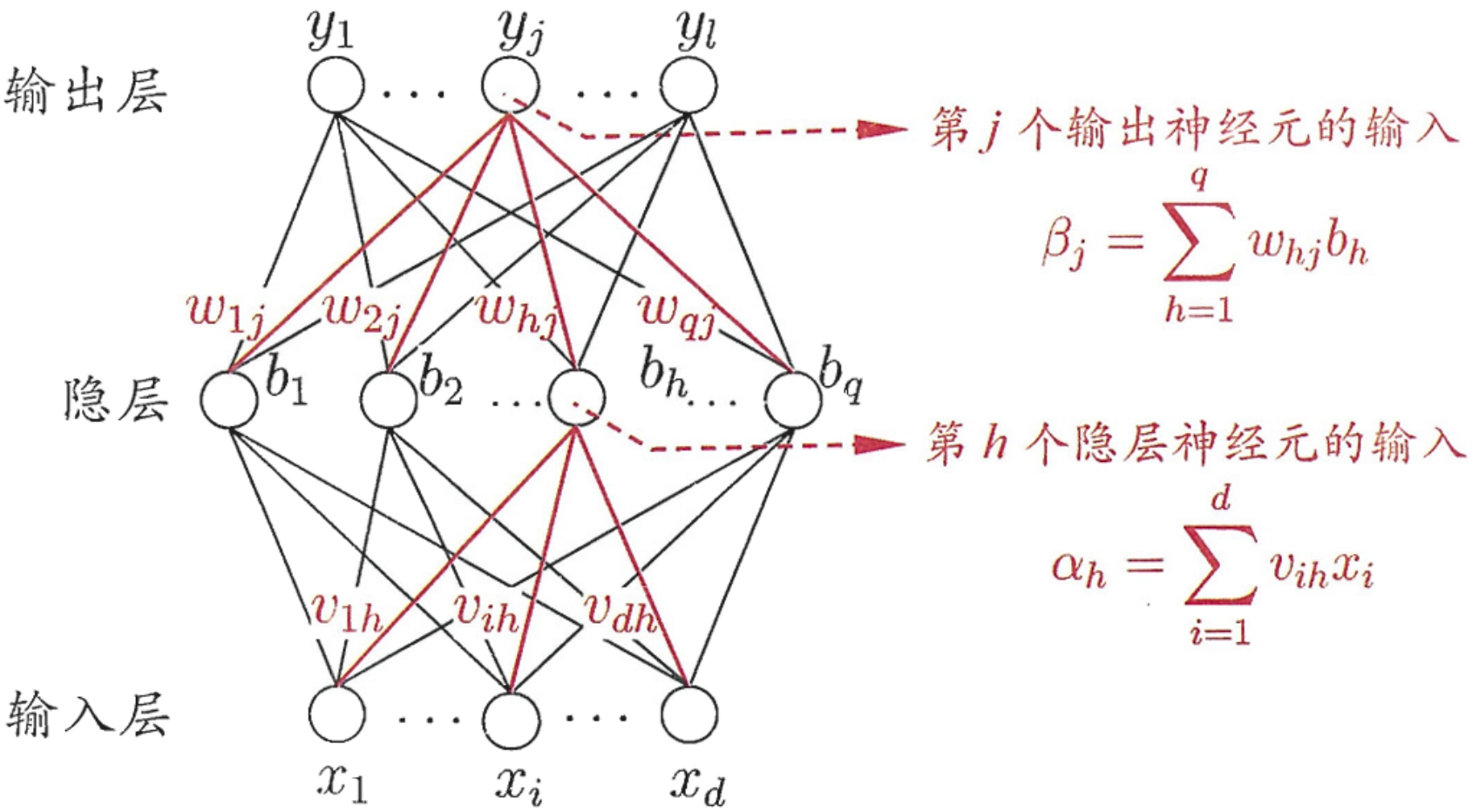
- 给定训练集$D=\{(\boldsymbol{x}_i,y_i)\},\boldsymbol{x}_i\in R^d,y_i\in R^l,(i=1,2,\cdots,m)$, 即输入示例由$d$个属性描述, 输出$l$ 维实值向量.
- 为方便讨论, 给定一个拥有 d个输入神经元,l 个输出神经元, q 个隐层神经元的多层前向前馈网络结构.
- 记号：

θ_j ： 输出层第j个神经元阈值；

γ_h：隐含层第 h 个神经元阈值；

v_{ih}：输入层与隐层神经元之间的连接权重；

w_{hj} ： 隐层与输出层神经元之间的连接权重；


输出层
隐层
输入层
y_1
y_j
y_l
w_{1j}
w_{2j}
w_{hj}
w_{qj}
b_1
b_2
b_h
b_q
v_{1h}
v_{ih}
v_{dh}
x_1
x_i
x_d
第 j 个输出神经元的输入
$\beta_j = \sum_{h=1}^{q} w_{hj} b_h$
第 h 个隐层神经元的输入
$\alpha_h = \sum_{i=1}^{d} v_{ih} x_i$

5.3 误差逆传播算法

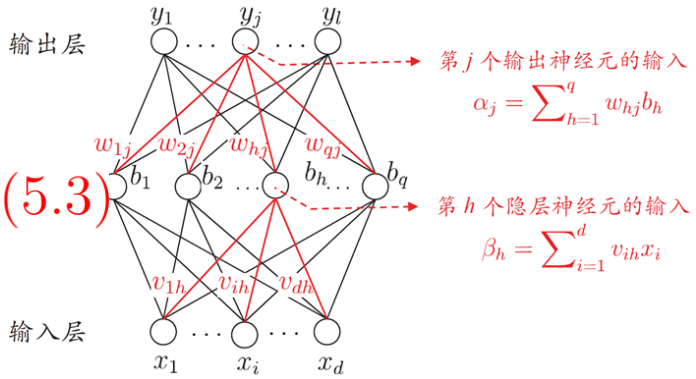
对于样例 $(\boldsymbol{x}_k, \boldsymbol{y}_k)$, 假设网络的实际输出为 $\hat{\boldsymbol{y}}_k$

- **前向计算**

step1: $b_h = f(\beta_h - \gamma_h), \beta_h = \sum_{i=1}^{d} v_{ih} x_i$

step2: $\hat{y}_j^k = f(\alpha_j - \theta_j), \alpha_h = \sum_{i=1}^{q} w_{hj} b_h$

step3: $E_k = \frac{1}{2} \sum_{j=1}^{l} (\hat{y}_j^k - y_j^k)^2$



- **参数数目**

权重:v_{ih} , w_{hj} 阈值: θ_j , γ_h $(i = 1, \cdots, d, h = 1, , \cdots, q, j = 1, \cdots, l)$

因此网络中需要$(d + l + 1)q + l$参数需要优化

- **参数优化**

BP是一个迭代学习算法, 在迭代的每一轮中采用广义的感知机学习规则对参数进行更新估计, 任意的参数v的更新估计式为

$$v \leftarrow v + \triangle v.$$

5.3 误差逆传播算法

BP算法基于梯度下降策略, 以目标的负梯度方向对参数进行调整. 对误差 E_k , 给定学习率 η

$$\Delta w_{hj} = -\eta \frac{\partial E_k}{\partial w_{hj}}$$

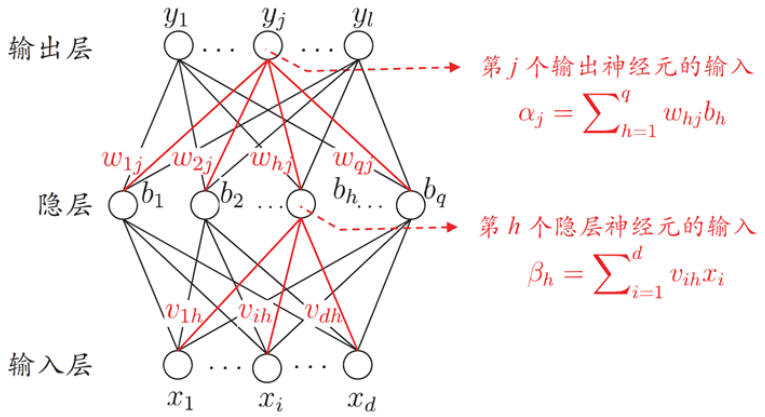
$$\frac{\partial E_k}{\partial w_{hj}} = \frac{\partial E_k}{\partial \hat{y}_j^k} \cdot \frac{\partial \hat{y}_j^k}{\partial \beta_j} \cdot \frac{\partial \beta_j}{\partial w_{hj}}$$

$$\begin{aligned} g_j &= -\frac{\partial E_k}{\partial \hat{y}_j^k} \cdot \frac{\partial \hat{y}_j^k}{\partial \beta_j} \\ &= -(\hat{y}_j^k - y_j^k) f'(\beta_j - \theta_j) \\ &= \hat{y}_j^k (1 - \hat{y}_j^k)(y_j^k - \hat{y}_j^k) \end{aligned} \quad (5.10)$$

$$\Delta w_{hj} = \eta g_j b_h \quad (5.11)$$

5.3 误差逆传播算法

BP 学习算法



- 类似的可以推导出：

$$\triangle \theta_j = -\eta g_j, \quad (5.12)$$

$$\triangle v_{ih} = \eta e_h x_i, \quad (5.13)$$

$$\triangle \gamma_h = -\eta e_h, \quad (5.14)$$

其中

$$e_h = -\frac{\partial E_k}{\partial b_h} \cdot \frac{\partial b_h}{\partial \alpha_h} = \sum_{j=1}^{h} w_{hj} g_j f'(\alpha_h - \gamma_h)$$

$$= -\sum_{j=1}^{l} \frac{\partial E_k}{\partial \beta_j} \partial b_h f'(\alpha_h - \gamma_h) = b_h(1 - b_h) \sum_{j=1}^{h} w_{hj} g_j. \quad (5.15)$$

- 学习率 $\eta \in (0, 1)$ 控制着算法每一轮迭代中的更新步长, 若太长则让容易震荡, 太小则收敛速度又会过慢.

5.3 误差逆传播算法

BP 学习算法

输入： 训练集 $D = \{(\boldsymbol{x}_k, \boldsymbol{y}_k)\}_{k=1}^{m}$;
学习率 η.

过程：

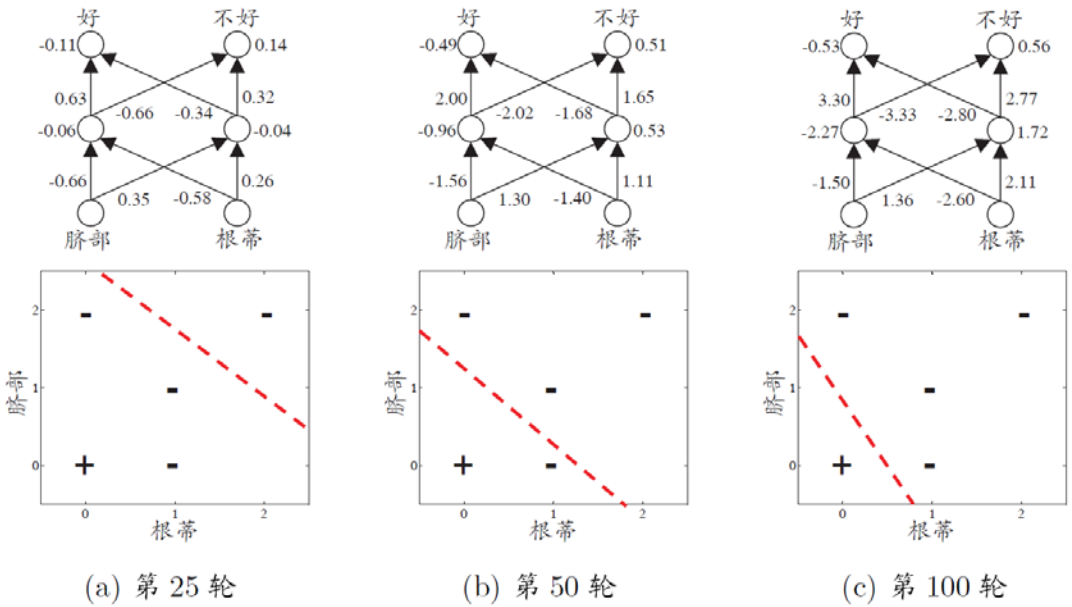
1: 在$(0,1)$范围内随机初始化网络中所有连接权和阈值
2: **repeat**
3: **for all** $(\boldsymbol{x}_k, \boldsymbol{y}_k) \in D$ **do**
4: 根据当前参数和式(5.3) 计算当前样本的输出 $\hat{\boldsymbol{y}}_k$;
5: 根据式(5.10) 计算输出层神经元的梯度项 g_j;
6: 根据式(5.15) 计算隐层神经元的梯度项 e_h;
7: 根据式(5.11)-(5.14) 更新连接权 w_{hj}, v_{ih} 与阈值 θ_j, γ_h
8: **end for**
9: **until** 达到停止条件

输出： 连接权与阈值确定的多层前馈神经网络

图 5.8 误差逆传播算法

5.3 误差逆传播算法

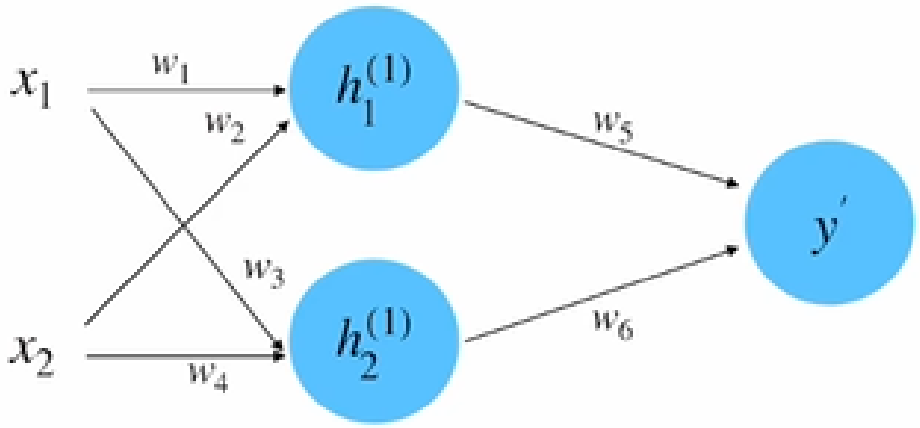
BP 算法实验



(a) 第 25 轮　(b) 第 50 轮　(c) 第 100 轮

图 5.9　在 2 个属性、5 个样本的西瓜数据上, BP网络参数更新和分类边界的变化情况

例子



假设这个样本的特征有两个维度，
每个维度的值为

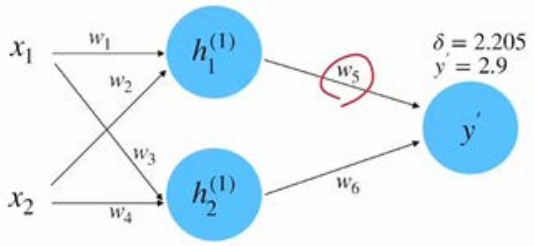
$$x_1 = 0.5$$
$$x_2 = 1.0$$

假设这个样本的真实值为：

$$y = 0.8$$

神经网络所有参数
随机初始化，且恰好如下

$$w_1 = 1.0$$
$$w_2 = 0.5$$
$$w_3 = 0.5$$
$$w_4 = 0.7$$
$$w_5 = 1.0$$
$$w_6 = 2.0$$



以计算w_5为例，w_6同理

$$\frac{\partial \delta}{\partial w_5} = \frac{\partial \delta}{\partial y'} \cdot \frac{\partial y'}{\partial w_5}$$

$$\delta = \frac{1}{2}(y - y')^2$$

$$\frac{\partial \delta}{\partial y'} = 2 \cdot \frac{1}{2} \cdot (y - y')(-1)$$

$$= y' - y$$

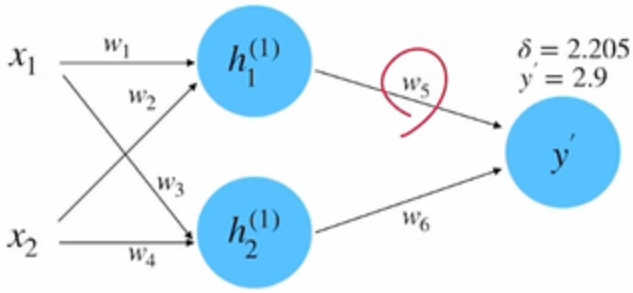
$$= 2.9 - 0.8 = 2.1$$

$$y' = w_5 \cdot h_1^{(1)} + w_6 \cdot h_2^{(1)}$$

$$\frac{\partial y'}{\partial w_5} = h_1^{(1)} + 0$$

$$= 1.0$$

$$\frac{\partial \delta}{\partial w_5} = 2.1 \cdot 1.0 = 2.1$$



由前一页可知

$$\frac{\partial \delta}{\partial w_5} = 2.1 \cdot 1.0 = 2.1$$

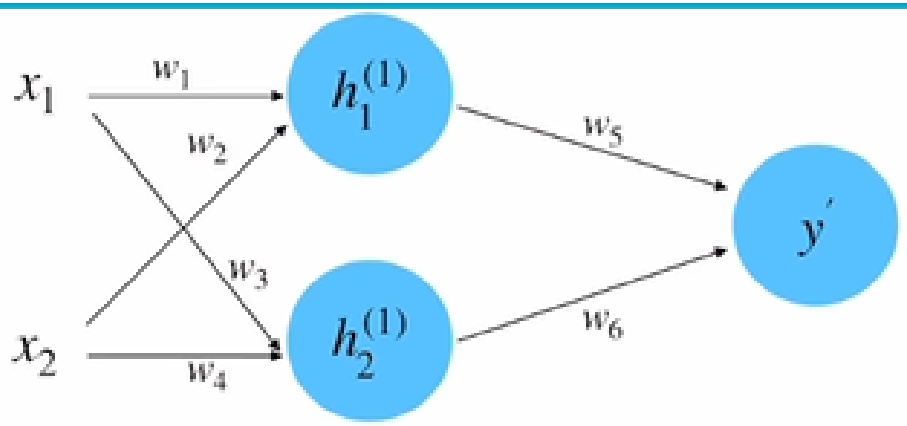
那么便可更新w5的值

学习率，属于超参数，自行设定

$$w_5^{(update)} = w_5 - \eta \cdot \frac{\partial \delta}{\partial w_5}$$

$$= 1.0 - 0.1 \cdot 2.1$$

$$= 0.79$$



假设这个样本的特征有两个维度，每个维度的值为

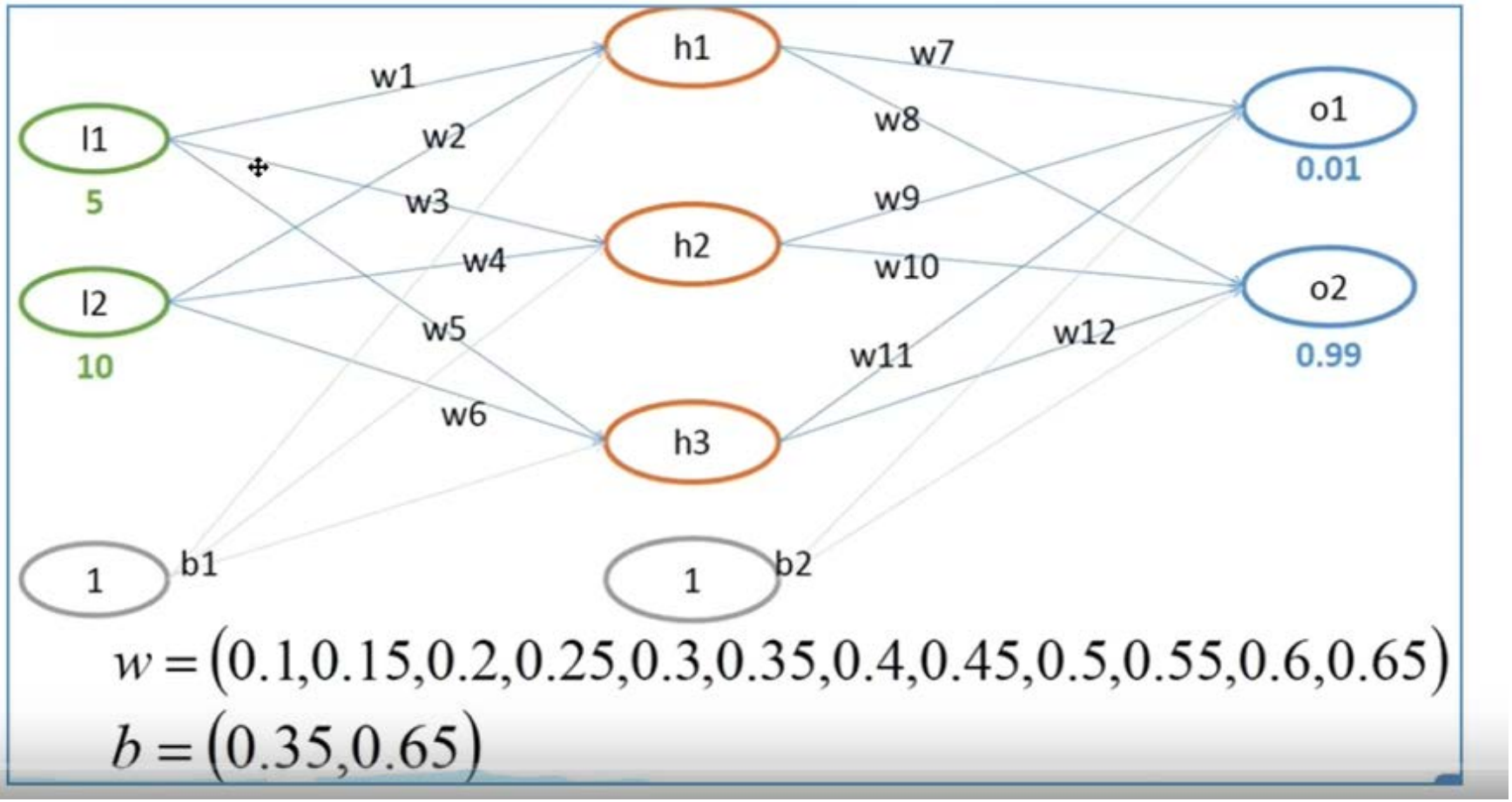
$$x_1 = 0.5$$
$$x_2 = 1.0$$

假设这个样本的真实值为：

$$y = 0.8$$

一轮正向反向传播后更新得到的参数：

$$w_1 = 0.895$$
$$w_2 = 0.29$$
$$w_3 = 0.29$$
$$w_4 = 0.28$$
$$w_5 = 0.79$$
$$w_6 = 1.8005$$

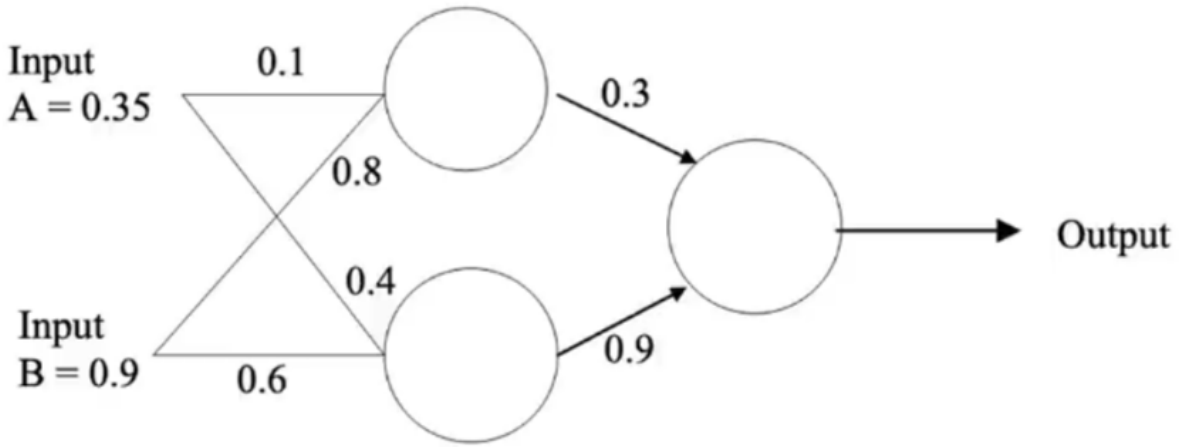


【神经网络】BP神经网络的权值来实现学习

练习题5.3

考虑下面的简单网络，假设隐藏层和输出层都使用 Sigmoid 激活函数，

学习率为 0.5，使用 SSE 误差，真实响应 y=0.5。



1. 使用前馈过程计算预测输出，并计算误差值；

2. 执行一次反向传播；

3. 再使用前馈过程并更新预测结果。

5.3 误差逆传播算法

- 标准 BP 算法
- 每次针对单个训练样例更新权值与阈值.
- 参数更新频繁, 不同样例可能抵消, 需要多次迭代.
- 累计 BP 算法
- 其优化的目标是最小化整个训练集上的累计误差

$$E = \frac{1}{m}\sum_{k=1}^{m} E_k$$

- 读取整个训练集一遍才对参数进行更新, 参数更新频率较低.
- 实际应用

但在很多任务中, 累计误差下降到一定程度后, 进一步下降会非常缓慢, 这时标准BP算法往往会获得较好的解, 尤其当训练集非常大时效果更明显.

5.3 误差逆传播算法

- ## 多层前馈网络表示能力

只需要一个包含足够多神经元的隐层，多层前馈神经网络就能以任意精度逼近任意复杂度的连续函数。

[Hornik et al.，1989]

- ## 多层前馈网络局限

- 神经网络由于强大的表示能力，经常遭遇过拟合. 表现为：训练误差持续降低，但测试误差却可能上升
- 如何设置隐层神经元的个数仍然是个未决问题. 实际应用中通常使用“试错法”调整

- ## 缓解过拟合的策略

- 早停：在训练过程中，若训练误差降低，但验证误差升高，则停止训练
- 正则化：在误差目标函数中增加一项描述网络复杂程度的部分，例如连接权值与阈值的平方和

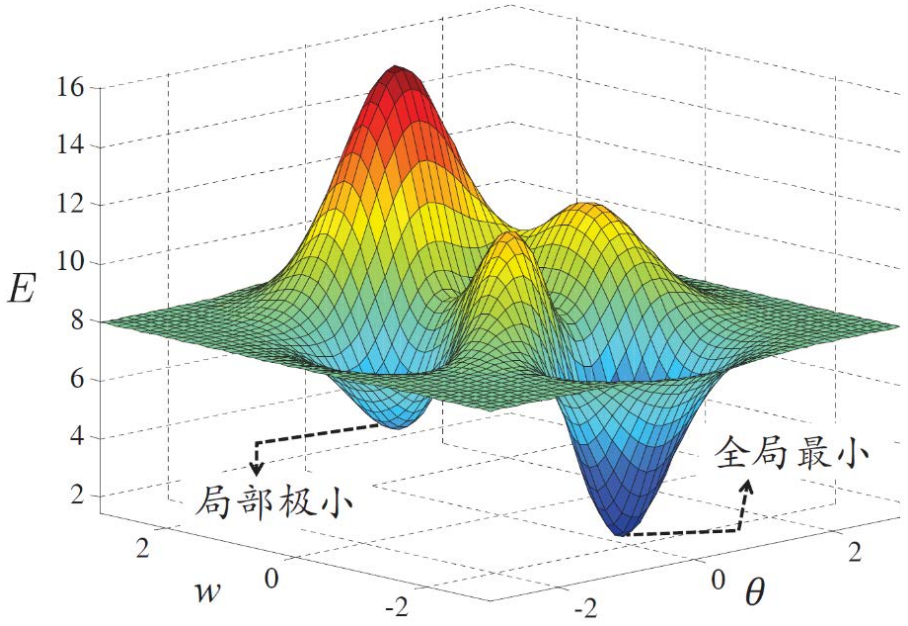
5.4 全局最小与局部极小

- 对$\boldsymbol{w}^*$和θ^*，若存在 $\epsilon > 0$ 使得

$$\forall(\boldsymbol{w};\theta) \in \{\|(\boldsymbol{w};\theta) - (\boldsymbol{w}^*;\theta^*)\| \leq \epsilon\},$$

都有$E(\boldsymbol{w};\theta) \geq E(\boldsymbol{w}^*;\theta^*)$成立，则$(\boldsymbol{w}^*;\theta^*)$为局部极小解；若度参数空间中任意的$(\boldsymbol{w};\theta)$，都有$E(\boldsymbol{w};\theta) \geq E(\boldsymbol{w}^*;\theta^*)$，则$(\boldsymbol{w}^*;\theta^*)$为全局最小解. 两者对应的$E(\boldsymbol{w}^*;\theta^*)$分别称为误差函数的局部最小解和全局最小值.

- 显然参数空间梯度为零的点，只要气误差函数值小于邻点的误差函数值，就是局部极小点
- 可能存在多个局部极小值，但却只会有一个全局极最小值



需要注意的是，在训练神经网络前，要将数据归一化，以便进行训练和分析。另外，对于BP神经网络，初始权值的设定对于网络训练有较大的影响。如果初始值的设定较大，使得加权后的输入落在激活函数的饱和区，会导致网络的权值调整过程缓慢，因此通常将初始值设定为0附近的小随机数。影响BP学习算法训练效率的另一个关键因素是学习率，学习率太小会导致训练时间过长，学习率太大可能会在误差减小过程中产生振荡，导致学习过程不收敛，因此在BP神经网络的设计中，倾向于选取较小的学习率以保证系统的稳定性，一般在0.01~0.8之间。

BP神经网络的输入层和输出层的节点数可以根据数据集的特性进行设定，但对于隐藏层数和隐藏层节点数的选取一般依赖于经验和背景知识。若隐藏层的节点数过少，网络就不能充分提取数据的非线性特征，这会使网络的误差较大；若隐藏层的节点数过多，不仅会使网络的训练时间过长，还容易出现过拟合现象，即网络的测试误差很大。在设计神经网络时，一般先考虑设置一个隐藏层，当一个隐藏层的节点数很多仍不能改善网络的性能时，可以考虑增加隐藏层，增加隐藏层的个数可以提高网络的非线性映射能力，进一步降低误差，提高训练精度，但加大隐藏层个数必定使训练过程复杂，训练时间延长。

5.4 全局最小与局部极小

- “跳出”局部最小的策略

基于梯度的搜索是使用最为广泛的参数寻优方法. 如果误差函数仅有一个局部极小，那么此时找到的局部极小就是全局最小；然而, 如果误差函数具有多个局部极小，则不能保证找到的解是全局最小. 在现实任务中，通常采用以下策略“跳出”局部极小，从而进一步达到全局最小.

- 多组不同的初始参数优化神经网络，选取误差最小的解作为最终参数.
- 模拟退火技术 [Aarts and Korst，1989]. 每一步都以一定的概率接受比当前解更差的结果, 从而有助于跳出局部极小.
- 随机梯度下降. 与标准梯度下降法精确计算梯度不同, 随机梯度下降法在计算梯度时加入了随机因素.
- 遗传算法 [Goldberg，1989]. 遗传算法也常用来训练神经网络以更好地逼近全局极小.

5.5 其他常见神经网络

RBF 网络 [Broomhead and Lowe, 1988]

- RBF 网络是一种单隐层前馈神经网络, 它使用径向基函数作为隐层神经元激活函数, 而输出层则是隐层神经元输出的线性组合.

- RBF网络模型

假定输入为d维的向量$\boldsymbol{x}$, 输出为实值, 则RBF 网络可以表示为

$$\varphi(\boldsymbol{x}) = \sum_{i=1}^{q} w_i \rho(\boldsymbol{x}, \boldsymbol{c}_i)$$

其中q为隐层神经元的个数,$\boldsymbol{c}_i$和w_i分别是第i神经元对应的中心和权重,$\rho(\boldsymbol{x}, \boldsymbol{c}_i)$是径向基函数.

常用的高斯径向基函数形如 $\rho(\boldsymbol{x}, \boldsymbol{c}_i) = e^{-\beta_i \|\boldsymbol{x}-\boldsymbol{c}_i\|^2}$

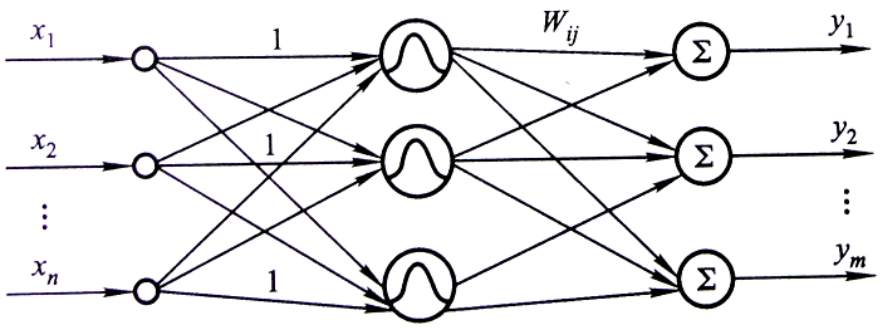
径向基函数（RBF，Radial Basis Function）神经网络是一种局部神经网络。其生物学基础是大脑皮层和视觉皮层的局部调节和交叉重叠的接受域。

与BP网络主要的不同是隐含层网络节点的激活函数用了径向基函数。

1. RBF网络结构

RBF网络的理论基础是函数逼近，用一个具有单隐层的前向网络来逼近任意函数。它可以任意精度逼近任意函数。

隐含层的激活函数实现了输入输出的非线性映射。输出层对隐含层的输出进行线性加权组合，属于线性变换。



RBF网络代表一类采用具有中心点的激活函数的神经网络。

$$\text{RBF}: b_j(x) = \phi_j(\|\boldsymbol{x} - \boldsymbol{c}_j\|), \quad j = 1, 2, \cdots, p$$

$b_j(x)$是隐含层输出；$\phi_j(\bullet)$是基函数，$\|\bullet\|$表示欧氏范数；c_j是基函数的中心点。以输入空间的点$\boldsymbol{x}$与中心$\boldsymbol{c}_j$的距离作为函数的自变量。因距离是径向同性的，固函数得名径向基函数。

通常采用高斯型的基函数。

$$b_j(x) = \exp\left(-\frac{\|\boldsymbol{x} - \boldsymbol{c}_j\|^2}{2\sigma_j^2}\right)$$

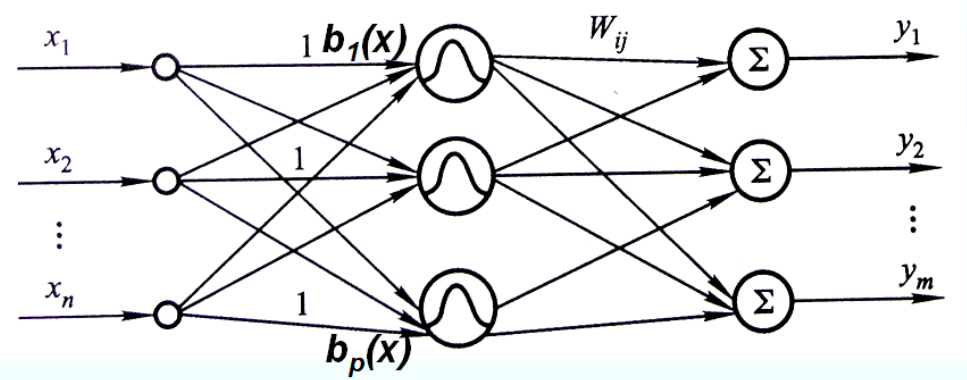
高斯函数是具有局部分布，对中心点径向对称衰减，非负的非线性函数。σ_j是扩展常数或宽度。σ_j越小，函数宽度越小。

输入信号靠近中心点$\boldsymbol{c}_j$时，隐含层才有较大的输出，具有局部化接收域的能力。

2. 网络输出计算

隐含层：$$b_j(x)=\exp\left(-\frac{\left\|\boldsymbol{x}-\boldsymbol{c}_j\right\|^2}{2\sigma_j^2}\right),\quad j=1,2,\cdots,p$$

输出层：$$y_i=\sum_{j=1}^{\mathrm{m}} w_{ij}b_j(\boldsymbol{x}),\quad i=1,2,\cdots,m$$



3. RBF网络的训练

1）基函数固定

根据输入样本随机选取基函数的中心点，如果基函数个数足够多，就能充分反映输入样本的概率分布，网络学习的结果能很好地逼近非线性函数。

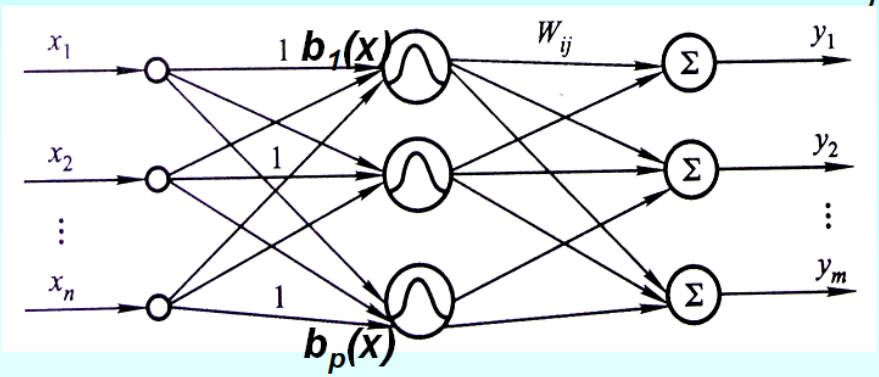
连接权的修正公式：

$$w_{ij}[k+1] = w_{ij}[k] + \beta(y_i^* - y_i[k])b_j(\boldsymbol{x}) / \boldsymbol{b}(\boldsymbol{x})^T \boldsymbol{b}(\boldsymbol{x})$$

基函数固定的RBF网络一般需要更多的基函数才能实现对较为复杂的非线性函数的逼近，网络规模一般较大。

2）基函数与权值同时调整

RBF网络调整的参数分别是中心向量c_j、RBF的宽度σ_j、和网络的连接权值。



分为两个阶段：

a. 由输入样本用聚类分析方法求出隐含层各节点的c_j和 σ_j

b. 由训练样本利用有教师学习算法训练调整连接权值。

① 无教师学习

对所有输入样本进行聚类，求得各隐层节点RBF的中心。用*k*-均值聚类算法调整中心。

算法步骤：

（1）随机给定各隐节点的初始中心 $\mathbf{c}_j(0)$，$j=1,2,\cdots,p$

（2）计算欧氏距离，并求出最小距离的节点

$$d_j(k)=\left\|\boldsymbol{x}(k)-\mathbf{c}_j(k-1)\right\|,1\le j\le p$$

（3）调整中心 $d_{\min}(k)=\min d_j(k)=d_r(k)$，$r$：隐层节点序号

$$\mathbf{c}_j(k)=\mathbf{c}_j(k-1)\ ,\ 1\le j\le p, j\ne r$$

$$\mathbf{c}_r(k)=\mathbf{c}_r(k-1)+\beta(\boldsymbol{x}(k)-\mathbf{c}_r(k-1))$$

β：学习速率，$0<\beta<1$

（4）***k=k+1***，转到第(2)步,重复上述过程，直到***c(k)***的改变量小于要求的值。

（5）确定扩展常数

$$\boldsymbol{d}_j(k)=\min_{i\ne j}\left\|\boldsymbol{c}_j-\mathbf{c}_i\right\| \qquad \sigma_j=\lambda\boldsymbol{d}_j$$

② 有教师学习

当 $\mathbf{c}_j$ 确定后，训练由隐层至输出层间的连接权值，它是线性方程组，则求连接权值成为线性优化问题，可利用各种线性优化算法求得，如：LMS算法、最小二乘递推法等。

设有 q 组输入/输出样本 $\boldsymbol{x}_q/\boldsymbol{d}_q, q=1,2,\ldots,L$。

目标函数

$$J=\sum_{q=1}^{L}\left\|\boldsymbol{e}_q\right\|_2=\sum_{q=1}^{L}\left\|\boldsymbol{d}_q-\boldsymbol{y}_q\right\|_2=\frac{1}{2}\sum_{q=1}^{L}\sum_{i=1}^{m}\left(d_{iq}-y_{iq}\right)^2$$

应用**LMS**，连接权值调整

$$w_{ji}(k+1)=w_{ji}(k)+\alpha\frac{e_q(k)\cdot x_{jq}}{\left\|x_q\right\|_2}$$

当 $J\le\varepsilon$ 算法结束。

RBF网络训练通用的方法是用误差反传机制，利用随机梯度法对基函数的中心和权值同时进行调整。

假设RBF网络是单输出，k 时刻的性能指标

$$E[k]=\frac{1}{2}\left|e[k]\right|^2=\frac{1}{2}\left[y^*[k]-\sum_j^M w_j[k]b(\boldsymbol{x}[k],\boldsymbol{c}_j[k])\right]^2$$

代入高斯函数

$$E[k]=\frac{1}{2}\left|e[k]\right|^2=\frac{1}{2}\left[y^*[k]-\sum_j^M w_j[k]\exp\left(-\frac{\left\|\boldsymbol{x}[k]-\boldsymbol{c}_j[k]\right\|^2}{\sigma_j^2[k]}\right)\right]^2$$

按随机梯度法得到参数修正公式

$$\boldsymbol{w}[k+1]=\boldsymbol{w}[k]-\eta_w\frac{\partial}{\partial \boldsymbol{w}}E[k]\Big|_{w=w[k]}=\boldsymbol{w}[k]+\eta_w e[k]\boldsymbol{b}[k]$$

$$\begin{aligned}\boldsymbol{c}_j[k+1]&=\boldsymbol{c}_j[k]-\eta_c\frac{\partial}{\partial \boldsymbol{c}_j}E[k]\Big|_{c_j=c_j[k]}\\&=\boldsymbol{c}_j[k]-\eta_c\frac{e[k]w_j[k]}{\sigma_j^2[k]}b(\boldsymbol{x}[k],\boldsymbol{c}_j[k],\sigma_j[k])(\boldsymbol{x}[k]-\boldsymbol{c}_j[k])\end{aligned}$$

$$\begin{aligned}\sigma_j[k+1]&=\sigma_j[k]-\eta_\sigma\frac{\partial}{\partial \sigma_j}E[k]\Big|_{\sigma_j=\sigma_j[k]}\\&=\sigma_j[k]-\eta_\sigma\frac{e[k]w_j[k]}{\sigma_j^2[k]}b(\boldsymbol{x}[k],\boldsymbol{c}_j[k],\sigma_j[k])\left\|\boldsymbol{x}[k]-\boldsymbol{c}_j[k]\right\|^2\end{aligned}$$

5.5 其他常见神经网络

RBF 网络

RBF网络性质

具有足够多隐层神经元RBF 神经网络能以任意精度逼近任意连续函数.

[Park and Sandberg, 1991]

RBF网络训练

- Step1:确定神经元中心，常用的方式包括随机采样、聚类等
- Step2:利用BP算法等确定参数

5.5 其他常见神经网络

Rprop神经网络

BP神经网络存在学习算法收敛速度慢、容易陷入局部极小值等缺陷。针对BP神经网络存在的问题，学者们提出了很多改进方法如：附加动量法、自适应学习速率法、拟牛顿法、共轭梯度法、LM算法等，其中比较经典的是Rprop（Resilient back propagation，弹性反向传播算法）神经网络。

由于误差超曲面非常复杂，很难从全局获得更多的启发式信息，所以可以考虑对每个权重分别采用不同的自适应学习率，即局部自适应策略。Riedmiller和Braun于1993年提出的Rprop算法是在前向神经网络中实现监督学习的局部自适应方案。

设w_{ij}是联系神经元i和j神经元之间的权重，E是可微的误差函数，上标代表迭代次数。在Rprop算法中，消除了偏导数的大小对权值改变的影响，权重更新的方向仅基于偏导数的符号，由专门的弹性更新值或步长（step-size）来确定，每个权重都有相对应的步长记作Δ_{ij}。典型的Rprop算法包括两个部分。

第一部分：调整step-size:

$$\Delta_{ij}^{(t)}=\begin{cases}\eta^{+}\Delta_{ij}^{(t-1)} & if \dfrac{\partial E}{\partial w_{ij}}^{(t-1)} \cdot \dfrac{\partial E}{\partial w_{ij}}^{(t)} > 0 \\ \eta^{-}\Delta_{ij}^{(t-1)} & if \dfrac{\partial E}{\partial w_{ij}}^{(t-1)} \cdot \dfrac{\partial E}{\partial w_{ij}}^{(t)} < 0 \\ \Delta_{ij}^{(t-1)} & else \end{cases}$$

这里 $0<\eta^{-}<1<\eta^{+}$，通常取 $\eta^{-}=0.5$ $\eta^{+}=1.2$ 步长是有界的，上下界记作Δ_{min}和Δ_{max}

第二部分：调整权重。这里有两种方法，Rprop+和Rprop-。

首先介绍Rprop+（Rprop with Weight-Backtracking）方法。

（a）如果 $\frac{\partial E}{\partial w_{ij}}^{(t-1)} \cdot \frac{\partial E}{\partial w_{ij}}^{(t)} > 0$，那么

$$\Delta_{ij}^{(t)} = \min(\eta^{+}\Delta_{ij}^{(t-1)}, \Delta_{\max})$$

$$\Delta w_{ij}^{(t)} = -sign(\frac{\partial E}{\partial w_{ij}}^{(t)})\Delta_{ij}^{(t)}$$

$$w_{ij}^{(t+1)} = w_{ij}^{(t)} + \Delta w_{ij}^{(t)}$$

（b）如果 $\frac{\partial E}{\partial w_{ij}}^{(t-1)} \cdot \frac{\partial E}{\partial w_{ij}}^{(t)} < 0$，那么 $\Delta_{ij}^{(t)} = \max(\eta^{-}\Delta_{ij}^{(t-1)}, \Delta_{\min})$

$$\Delta w_{ij}^{(t)} = -\Delta w_{ij}^{(t-1)}$$

$$w_{ij}^{(t+1)} = w_{ij}^{(t)} - \Delta w_{ij}^{(t-1)}$$

令 $\frac{\partial E}{\partial w_{ij}}^{(t)} = 0$（相当于强制设定 $\frac{\partial E}{\partial w_{ij}}^{(t)} = 0$ ）

（c）如果 $\frac{\partial E}{\partial w_{ij}}^{(t-1)} \cdot \frac{\partial E}{\partial w_{ij}}^{(t)} = 0$ ，那么

$$\Delta w_{ij}^{(t)} = -sign(\frac{\partial E}{\partial w_{ij}}^{(t)})\Delta_{ij}^{(t)}$$

$$w_{ij}^{(t+1)} = w_{ij}^{(t)} + \Delta w_{ij}^{(t)}$$

权重回溯（Weight-Backtracking）指的是如果前后两次误差函数的梯度符号改变说明前一次的更新太大，跳过了一个局部极小值点，那么令

$$\Delta w_{ij}^{(t)} = -\Delta w_{ij}^{(t-1)}$$

$$\frac{\partial E}{\partial w_{ij}}^{(t)} = 0$$

接着介绍Rprop-（Rprop without Weight-Backtracking）。

（a）如果 $\frac{\partial E}{\partial w_{ij}}^{(t-1)} \cdot \frac{\partial E}{\partial w_{ij}}^{(t)} > 0$ 那么

$$\Delta_{ij}^{(t)} = \min(\eta^{+}\Delta_{ij}^{(t-1)}, \Delta_{\max})$$

（b）如果 $\frac{\partial E}{\partial w_{ij}}^{(t-1)} \cdot \frac{\partial E}{\partial w_{ij}}^{(t)} < 0$ 那么

$$\Delta_{ij}^{(t)} = \max(\eta^{-}\Delta_{ij}^{(t-1)}, \Delta_{\min})$$

权重更新为 $w_{ij}^{(t+1)} = w_{ij}^{(t)} - sign(\frac{\partial E}{\partial w_{ij}}^{(t)})\Delta_{ij}^{(t)}$

RPROP方法只是根据梯度的方向来决定权重调整的方向，而不考虑梯度值的大小，因此不会受到由于不可预见的干扰导致的梯度坏值的影响。由于除了梯度的计算以外，权重的计算实际上也只依赖于弹性更新值（Resilient）的计算，而弹性更新值的计算很简单，因此计算量要比很多其他算法小。此外，一般的调整方法由于激活函数的限制，距离输出层越远的权重调整的越慢，学习能力越弱。由于RPROP算法不受梯度值大小的影响，在各层网络都具有相同的学习能力，而不受与输出层距离的影响。

5.5 其他常见神经网络

ART 网络

- 竞争学习

竞争学习是神经网络中一种常用的无监督学习策略, 在使用该策略时, 网络的输出神经元相互竞争, 每一时刻仅有一个神经元被激活, 其他神经元的状态被抑制.

- ART 网络 [Carpenter and Grossberg, 1987]
- ART 网络是竞争学习的重要代表
- ART 网络由比较层、识别层、识别阈值和重置模块构成
- 比较层负责接收输入样本, 并将其传送给识别层神经元
- 识别层每个神经元对应一个模式类, 神经元的数目可在训练过程中动态增长以增加新的模式类

5.5 其他常见神经网络

ART 网络

- ART 网络性能依赖于识别阈值
- 识别阈值高时, 输入样本将会分成比较多、得到较精细分类
- 识别阈值低时, 输入样本将会分成比较少、产生较粗略分类

- ART 网络的优势
- ART较好的解决了竞争学习中的“可塑性-稳定性窘境”, 可塑性是指神经网络要有学习新知识的能力；稳定性是指神经网络在学习新知识时要保持对旧知识的记忆.
- ART 网络可以增量学习或在线学习

- ART 网络的发展

ART2 网络、FuzzyART 网络、ARTMAP 网络

5.5 其他常见神经网络

SOM 网络 [Kohonen, 1982]

- SOM 网络是一种竞争型的无监督神经网络, 它能将高维数据映射到低维空间（通常为2维）, 同时保持输入数据在高维空间的拓扑结构, 即将高维空间中相似的样本点映射到网络输出层中邻近神经元.
- 如图，SOM 网络中的输出层神经元以矩阵方式排列在二维空间中, 每个神经元都拥有一个权值向量, 网络在接收输入向量后, 将会确定输出层获胜神经元, 它决定了该输入向量在低维空间中的位置.

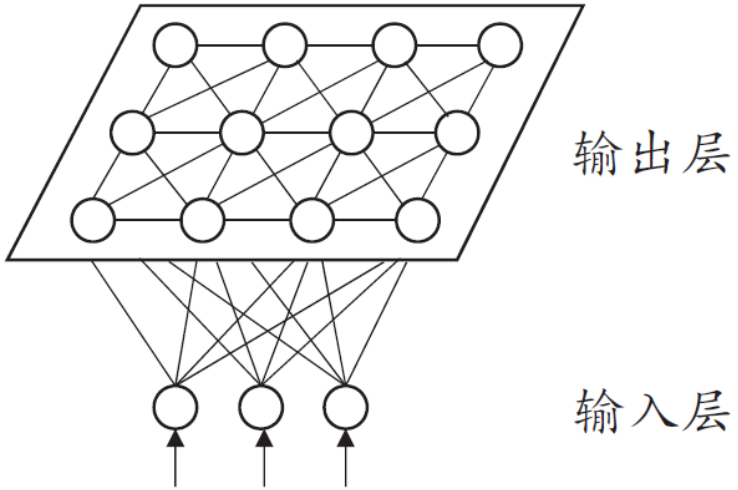


图 5.11 SOM 网络结构

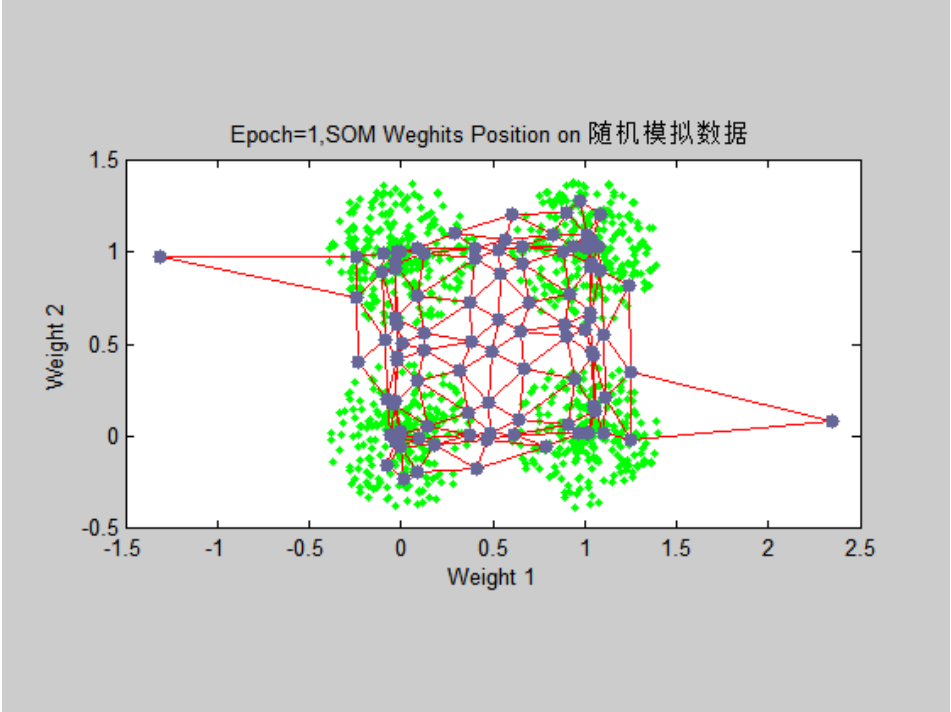
5.5 其他常见神经网络

SOM 网络 [Kohonen, 1982]

- SOM 网络训练

Step1:接受到一个训练样本后, 每个输出层神经元计算该样本与自身携带的权向量之间的距离, 距离最近的神经元成为竞争获胜者

Step2:最佳匹配单元及其近邻神经元的权值将被调整, 使得这些权向量与当前输入样本的距离缩小



5.5 其他常见神经网络

级联相关网络 [Fahlman and Lebiere 1990]

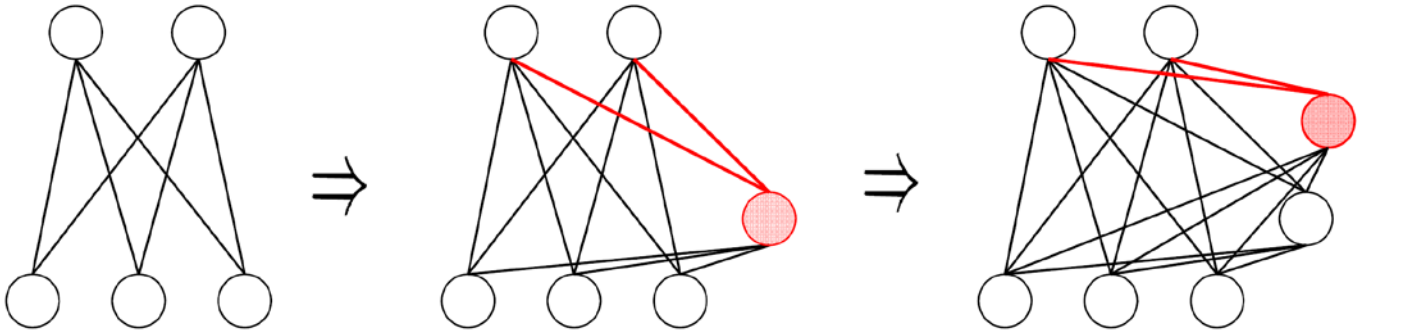
级联相关网络不仅利用训练样本优化连接权值, 阈值参数, 将网络的结构也当做学习的目标之一, 希望在训练过程中找到适合数据的网络结构.

- 级联与相关

级联：建立层次连接的层级结构

相关：最大化神经元的输出与网络误差时间的相关性来训练相关参数

- 网络优化演示



(a) 初始状态　(b) 增加一个隐层结点　(c) 增加第二个隐层结点

5.5 其他常见神经网络

Elman 网络 [Elman 1990]

- **递归神经网络**
  - 允许网络中出现环形结构, 使得神经元的输出反馈回来作为输入信号
  - *t* 时刻网络的输出状态：由 *t* 时刻的输入状态和 *t*-1 时刻的网络状态决定
- **Elman 网络**

Elamn 网络是最常用的递归神经网络之一, 结构如图所示, 这种结构与前馈神经网络很相似, 但是隐层神经元的输出被反馈回来, 与下一时刻输入层神经元提供的信号一起, 作为隐层神经元在下一时刻的输入

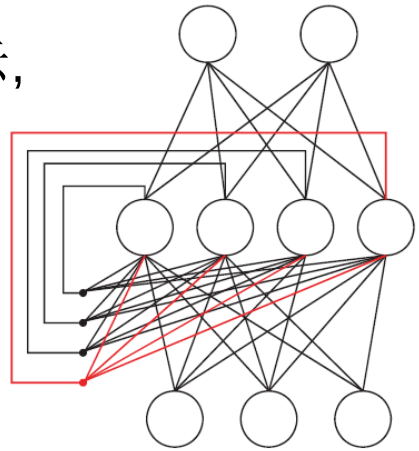


图 5.13 Elman 网络结构

- **训练算法**

推广的BP算法. [Pineda, 1987]

5.5 其他常见神经网络

Boltzmann 机

- **能量模型**

神经网络中有一类模型为网络定义一个“能量”，能量最小化时网络达到理想状态，而网络的训练就是在最小化这个能量函数.

- **Boltzmann 机**

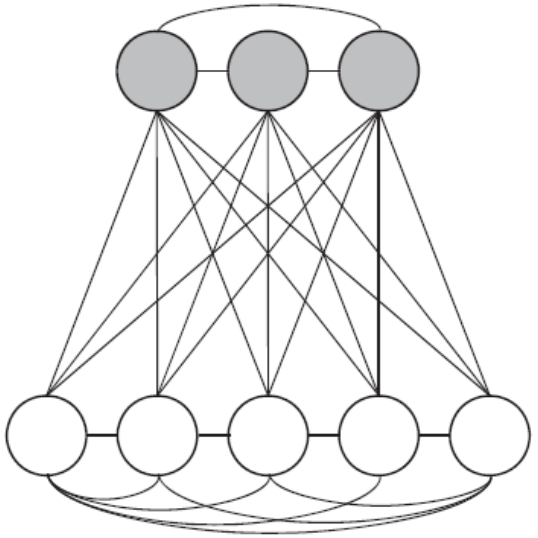
- Boltzmann 机就是一种基于能量的模型
- 结构：显层与隐层

  显层：数据的输入输出

  隐层：数据的内在表达
- 神经元

  布尔型，即只能取0和1 两种状态，

  其中1 表示激活，0表示抑制.



(a) Boltzmann机

5.5 其他常见神经网络

Boltzmann 机 [Ackley et al. , 1985]

- **Boltzmann 机能量**

令状态向量 $\boldsymbol{s} \in \{0,1\}^n$, 则其对应的Boltzmann 机能量定义为

$$E(\boldsymbol{s}) = -\sum_{i=1}^{n-1}\sum_{j=i+1}^{n} w_{ij}s_i s_j - \sum_{i=1}^{n}\theta_i s_i$$

其中 w_{ij} 表示两个神经元之间的连接权值, θ_i 表示神经元的阈值.

- **Boltzmann 分布**

网络中的神经元以任意不依赖与输入值得顺序进行更新，则网络最终将达到Boltzmann 分布，此时状态向量出现的概率将仅由其能量与所有可能状态向量的能量确定:

$$P(\boldsymbol{s}) = \frac{e^{-E(\boldsymbol{s})}}{\sum_t e^{-E(t)}}$$

5.5 其他常见神经网络

Boltzmann 机 [Ackley et al. , 1985]

- Boltzmann 机训练
  - 将每个训练样本视为一个状态向量，使其出现的概率尽可能大
  - 标准的 Boltzmann 机是一个全连接图，训练网络的复杂度很高，这使其难以用于解决现实任务
  - 现实中常用受限 Boltzmann 机，简称RBM. RBM 仅保留显层与隐层之间的连接，从而将Boltzmann机结构有完全图简化为二部图

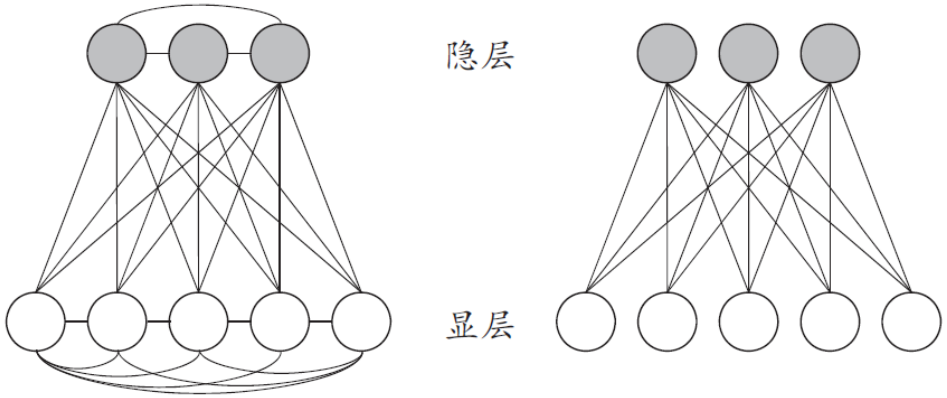


图 5.14 Boltzmann 机与受限 Boltzmann 机

5.5 其他常见神经网络

受限 Boltzmann 机 [Ackley et al. , 1985]

- 受限 Boltzmann 机常用“对比散度”（简称：CD）算法 [Hinton, 2010] 来进行训练
- 假定网络中有d 个显层神经元 q 个隐层神经元，令 $\boldsymbol{v}$和 $\boldsymbol{h}$分别是显层与隐层的状态向量，由于同一层内不存在连接，有

$$P(\boldsymbol{v}|\boldsymbol{h}) = -\prod_{i=1}^{d} P(v_i|\boldsymbol{h}),$$

$$P(\boldsymbol{h}|\boldsymbol{v}) = -\prod_{i=1}^{q} P(h_i|\boldsymbol{v}),$$

- CD算法对每个训练样本$\boldsymbol{v}$ 先计算出隐层神经元状态的概率分布，然后根据这个概率分布采样得到$\boldsymbol{h}$；类似的方法从$\boldsymbol{h}$ 中产生 $\boldsymbol{v}'$再从$\boldsymbol{v}'$ 中产生 $\boldsymbol{h}'$ 连接权重的更新公式为：

$$\triangle w = \boldsymbol{v}\boldsymbol{h}^T - \boldsymbol{v}'\boldsymbol{h}'^T$$

用神经网络作分类器

- 缺点
  - 训练时间长
  - 学习函数(权重)很难理解
  - 不容易包含论域知识
- 优点
  - 预测的准确率通常很高
  - 强壮性好,当训练样本包含错误时很有效
  - 输出可以是离散的,实数值的或几个离散或实数值属性的向量
  - 学习目标函数的快速评估

5.6 深度学习

深度学习模型

- 典型的深度学习模型就是很深层的神经网络.

●模型复杂度

- 增加隐层神经元的数目 (模型宽度)
- 增加隐层数目（模型深度）
- 从增加模型复杂度的角度看, 增加隐层的数目比增加隐层神经元的数目更有效. 这是因为增加隐层数不仅增加额拥有激活函数的神经元数目, 还增加了激活函数嵌套的层数.

●复杂模型难点

多隐层网络难以直接用经典算法（例如标准BP算法）进行训练, 因为误差在多隐层内逆传播时, 往往会”发散”而不能收敛到稳定状态.

5.6 深度学习

复杂模型训练方法

●预训练+微调

- **预训练**:监督逐层训练是多隐层网络训练的有效手段，每次训练一层隐层结点，训练时将上一层隐层结点的输出作为输入，而本层隐结点的输出作为下一层隐结点的输入，这称为"预训练".
- **微调：**在预训练全部完成后，再对整个网络进行微调训练. 微调一般使用BP算法.
- 例子： 深度信念网络 [Hintonet al.， 2006]
- 结构：每一层都是一个受限 Boltzmann 机
- 训练方法：无监督预训练+BP 微调
- 分析

预训练+微调 的做法可以视为将大量参数分组，对每组先找到局部看起来比较好的设置，然后再基于这些局部较优的结果联合起来进行全局寻优.

5.6 深度学习

复杂模型训练方法

●权共享

- 一组神经元使用相同的连接权值.
- 权共享策略在卷积神经网络(CNN)[LeCun and Bengio, 1995; LeCun et al., 1998]中发挥了重要作用.

●卷积神经网络

结构：CNN复合多个卷积层和采样层对输入信号进行加工, 然后在连接层实现与输出目标之间的映射.

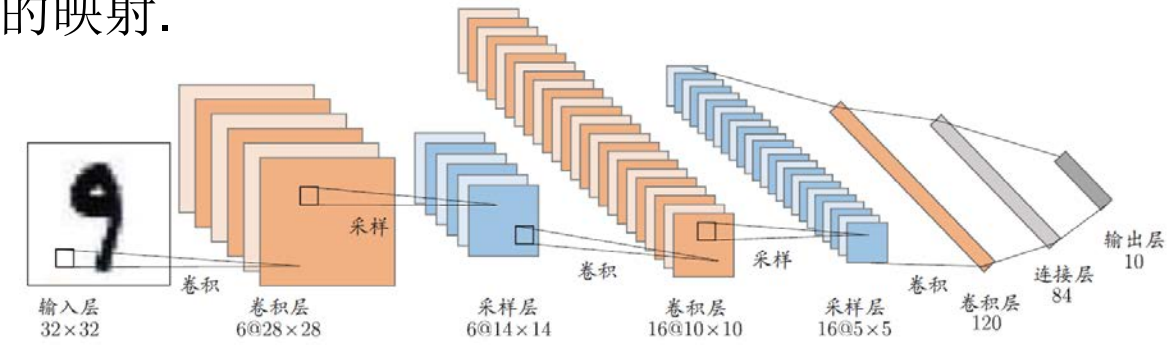


图 5.15 卷积神经网络用于手写数字识别 [LeCun et al., 1998]

5.6 深度学习

●卷积神经网络

- 卷基层：每个卷基层包含多个特征映射，每个特征映射是一个由多个神经元构成的“平面”，通过一种卷积滤波器提取的一种特征
- 采样层：亦称“汇合层”，其作用是基于局部相关性原理进行亚采样，从而在减少数据量的同时保留有用信息
- 连接层：每个神经元被全连接到上一层每个神经元，本质就是传统的神经网络，其目的是通过连接层和输出层的连接完成识别任务

●卷积神经网络激活函数

在CNN中通常将 sigmoid 激活函数替换为修正的线性函数

$$f(x) = \max(0, x)$$

●卷积神经网络训练

CNN 可以用BP进行训练，但在训练中，无论是卷积层还是采样层，每一组神经元都是用相同的连接权，从而大幅减少了需要训练的参数数目

5.6 深度学习

理解深度学习

- “特征工程” VS“特征学习”或者 “表示学习”
- 特征工程由人类专家根据现实任务来设计，特征提取与识别是单独的两个阶段



手工设计特征 → 分类识别

- 特征学习通过深度学习技术自动产生有益于分类的特征，是一个端到端的学习框架



学习生成特征 → 分类识别

5.6 深度学习

深度学习常用软件包

- CAFFE

  The Berkeley Vision and Learning Center (BVLC)

  http://caffe.berkeleyvision.org/

- MatConvNet

  The Oxford Visual Geometry Group (VGG)

  http://www.vlfeat.org/matconvnet/

- Torch

  http://torch.ch/

神经网络基础教材

http://hagan.okstate.edu/

5.7 阅读材料

主流学术期刊

- Neural Computation
- Neural Networks
- IEEE Transactions on Neural Networks and Learning Systems

国际会议

- 国际神经信息处理系统会议（NIPS）
- 国际神经网络联合会议（IJCNN）

区域性国际会议

- 欧洲神经网络会议（ICANN）
- 亚太神经网络会议（ICONIP）

谢谢大家！